HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO BÀI TẬP AGENT 01

ĐỀ TÀI: TÌM HIỂU VỀ HỆ TÁC TỬ THÔNG MINH (AI AGENT) VÀ TÁC TỬ HỌC TĂNG CƯỜNG (REINFORCEMENT LEARNING)

Lớp:	D23CTPM01
Nhóm:	Nhóm lớp CT – Nhóm 07
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 10/2026

Lời mở đầu và tổng quan bài tập

Các mô hình ngôn ngữ lớn (Large Language Models – LLM) đã thay đổi cách máy tính xử lý ngôn ngữ tự nhiên: chỉ với một câu lệnh, mô hình có thể tóm tắt văn bản, viết mã, trả lời câu hỏi chuyên môn. Nhưng một mô hình nền tảng thuần túy vẫn làm việc theo kiểu *hỏi – đáp* (prompt-and-response). Nó chờ người dùng hỏi rồi mới trả lời, tri thức dừng lại ở thời điểm huấn luyện, và nó không tự mở cơ sở dữ liệu, không tự gọi một API hay chạy một đoạn mã để thay đổi trạng thái của thế giới số.

Bước chuyển đang diễn ra là đưa mô hình nền tảng vào bên trong một **hệ tác tử thông minh** (AI Agent, Agentic AI): một hệ thống phần mềm tự đặt kế hoạch, tự chọn công cụ, ghi nhớ ngữ cảnh và hành động liên tục để đạt mục tiêu người dùng giao. Bài tập Agent 01 yêu cầu tìm hiểu khái niệm này từ ba nguồn tài liệu công nghệ: **Google Cloud** [1], **IBM Think** [2] và **McKinsey** [3]. Đồng thời, bài tập yêu cầu thực hành một dạng tác tử cổ điển là **tác tử học tăng cường** theo tài liệu **GeeksforGeeks** [4]: lập trình trên Jupyter Notebook, chụp lại màn hình mã nguồn (copy screen code) và màn hình kết quả (copy screen output).

Mục tiêu

1. Trình bày định nghĩa, ba trụ cột kỹ thuật và sự tiến hóa của AI Agent theo Google Cloud và McKinsey.
2. Phân tích chu trình nhận thức, các mô hình suy luận (ReAct, ReWOO và các mô hình khác), các loại tác tử, rủi ro và nguyên tắc quản trị theo IBM Think.
3. Trình bày nền tảng học tăng cường theo GeeksforGeeks: bài toán thăm dò – khai thác, chính sách ε-greedy, phương trình Bellman và các trường phái thuật toán.
4. Xây dựng tác tử Q-Learning trên Jupyter Notebook giải bài toán robot điều hướng mê cung, chụp màn hình mã và kết quả, rồi phân tích quá trình hội tụ của chính sách.
5. Liên hệ học tăng cường với các tác tử LLM hiện đại (RLHF, suy luận dạng

cây, hệ đa tác tử).

Bố cục báo cáo

Chương 1 và Chương 2 trình bày bản chất của AI Agent và lộ trình tiến hóa từ chatbot đến mạng lưới tác tử. Chương 3 đi vào kiến trúc kỹ thuật và các cơ chế suy luận theo IBM Think. Chương 4 quy chiếu tác tử hiện đại về năm loại tác tử kinh điển của Russell & Norvig [5] và khung Quá trình Quyết định Markov. Chương 5 trình bày lý thuyết học tăng cường. Chương 6 tổng hợp các nhóm ứng dụng doanh nghiệp cùng rủi ro và cách quản trị. Chương 7 và Chương 8 là phần thực nghiệm: mô tả bài toán, mã nguồn, ảnh chụp màn hình notebook và phân tích kết quả. Chương 9 bàn về sự giao thoa giữa LLM Agent và học tăng cường; Chương 10 tổng kết.

Cách tạo số liệu trong báo cáo

Notebook `agent01_rl_qlearning.ipynb` được sinh và chạy tự động từ mã nguồn trong gói `ag1`. Khi chạy xong, notebook ghi kết quả ra tệp JSON. Mọi con số thực nghiệm trong văn bản và bảng được đọc từ tệp này khi dựng báo cáo; các đoạn mã in trong báo cáo được trích nguyên văn từ chính mã đã chạy. Ảnh chụp màn hình là notebook đó mở trong JupyterLab.

Chương 1

Bản chất và ba trụ cột nền tảng của AI Agent

1.1 Định nghĩa học thuật theo Google Cloud và McKinsey

Google Cloud định nghĩa AI Agent là hệ thống phần mềm dùng trí tuệ nhân tạo để theo đuổi mục tiêu và hoàn thành nhiệm vụ thay mặt người dùng; tác tử thể hiện khả năng suy luận, lập kế hoạch và ghi nhớ, đồng thời có một mức độ tự chủ để tự ra quyết định, tự học và tự thích nghi [1]. McKinsey nhấn mạnh tác tử là hệ thống dựa trên mô hình nền tảng, có thể *thực thi* các quy trình nhiều bước phức tạp trong thế giới số: nhận một yêu cầu bằng ngôn ngữ tự nhiên, tự chia nhỏ thành các bước, gọi công cụ và hoàn thành công việc thay vì chỉ trả lời [3].

Định nghĩa làm việc dùng trong báo cáo

AI Agent là hệ thống phần mềm vận hành bởi mô hình AI tạo sinh, có mức độ tự trị để chủ động theo đuổi một mục tiêu: tự lập kế hoạch, tự duy trì ngữ cảnh và tự sử dụng công cụ bên ngoài để hoàn thành chuỗi tác vụ phức hợp thay mặt người dùng.

Hai đặc điểm phân biệt tác tử với một mô hình ngôn ngữ đứng riêng:

- **Tính chủ động hướng mục tiêu.** Người dùng giao *kết quả cần đạt* chứ không giao từng bước; tác tử tự quyết định trình tự hành động.
- **Khả năng tác động vào môi trường.** Tác tử không chỉ sinh văn bản mà còn đọc, ghi dữ liệu, gọi API và quan sát kết quả hành động của chính nó để điều chỉnh bước tiếp theo.

1.2 Ba trụ cột kỹ thuật của một tác tử thông minh

Tổng hợp từ Google Cloud và McKinsey [1, 3], một tác tử tiêu chuẩn được dựng trên ba trụ cột quanh lõi mô hình nền tảng (Hình 1.1).

1.2.1 Suy luận và lập kế hoạch (Reasoning & Planning)

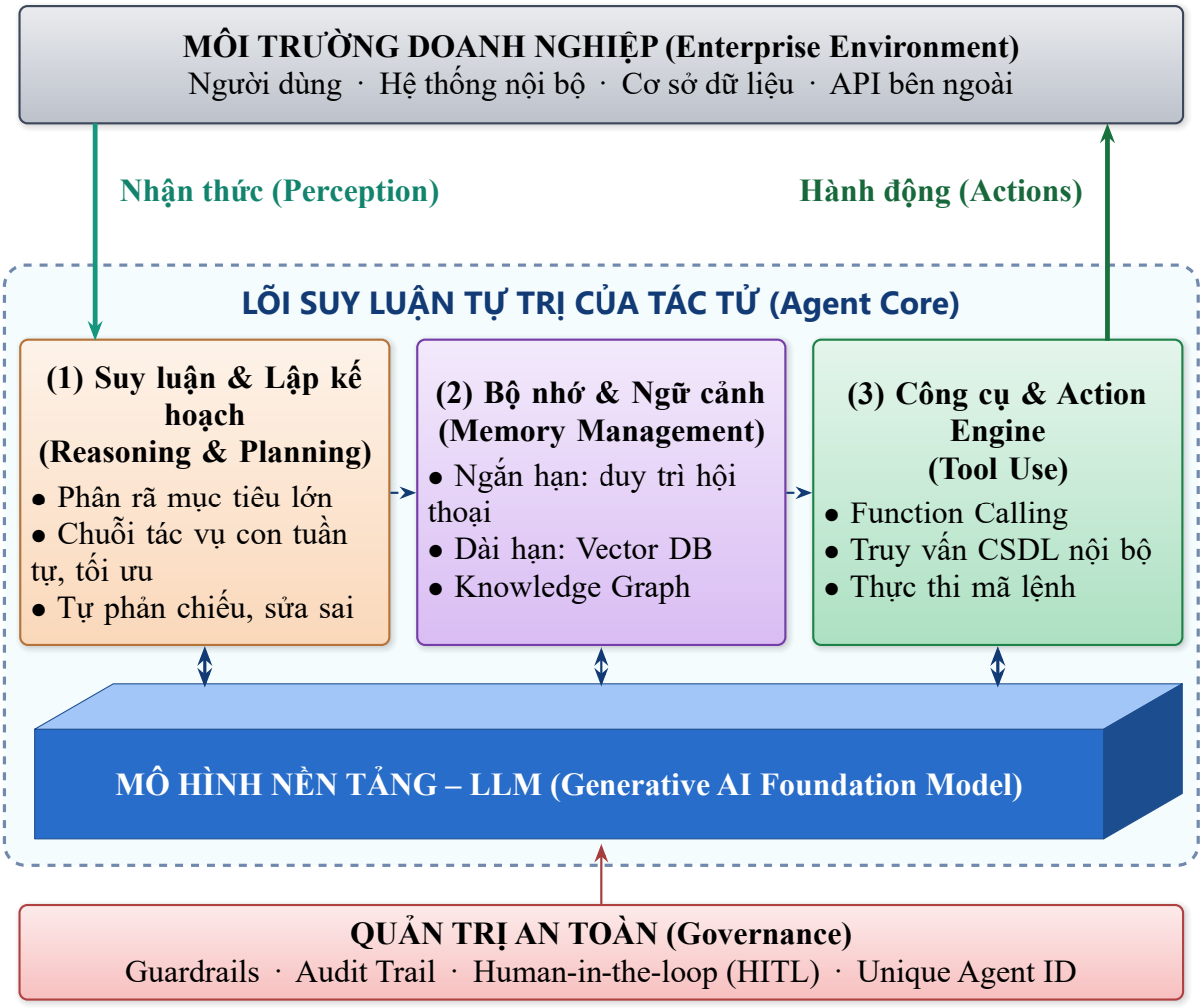
Tác tử phân rã mục tiêu lớn thành chuỗi tác vụ con có thứ tự, đánh giá các phương án và chọn trình tự hợp lý nhất. Mô hình nền tảng đóng vai bộ não suy luận: hiểu yêu cầu, lập kế hoạch, và khi kết quả trung gian không như mong đợi thì lập lại kế hoạch.

1.2.2 Bộ nhớ và quản lý ngữ cảnh (Memory Management)

- **Bộ nhớ ngắn hạn** giữ lịch sử hội thoại và trạng thái của phiên làm việc hiện tại, để tác tử hiểu các tham chiếu như "kết quả vừa rồi".
- **Bộ nhớ dài hạn** lưu tri thức và kinh nghiệm qua nhiều phiên, thường đặt trong cơ sở dữ liệu vector (Vector DB) hoặc đồ thị tri thức (Knowledge Graph) và được truy xuất khi cần.

1.2.3 Sử dụng công cụ ngoại vi (Tool Use & Action Engine)

Đây là trụ cột biến "lời nói" thành "hành động". Tác tử kích hoạt Function Calling để gọi API, truy vấn cơ sở dữ liệu nội bộ, chạy mã lệnh, tìm kiếm web. Kết quả công cụ trả về trở thành quan sát mới cho vòng suy luận tiếp theo.



Hình 1.1. Sơ đồ kiến trúc nhận thức của một hệ tác tử thông minh (AI Agent Cognitive Architecture): lõi mô hình nền tảng, ba trụ cột suy luận – bộ nhớ – công cụ, môi trường doanh nghiệp và lớp quản trị.

1.3 Vòng tương tác giữa ba trụ cột

Ba trụ cột không hoạt động rời rạc. Khi nhận mục tiêu, tác tử dùng trụ cột suy luận để lập kế hoạch; mỗi bước của kế hoạch có thể kích hoạt một công cụ; kết quả công cụ được ghi vào bộ nhớ ngắn hạn và đưa trở lại cho bước suy luận kế tiếp; tri thức đáng giữ được chuyển vào bộ nhớ dài hạn. Bảng 1.1 tóm tắt vai trò của từng trụ cột.

Bảng 1.1. Ba trụ cột kỹ thuật của AI Agent và vai trò của chúng.

Trụ cột	**Chức năng**	**Thành phần kỹ thuật điển hình**
Suy luận & lập kế hoạch	Phân rã mục tiêu, sắp xếp tác vụ con, lập lại kế hoạch khi gặp lỗi	LLM, kỹ thuật chuỗi suy nghĩ, mô hình ReAct/ReWOO
Bộ nhớ & ngữ cảnh	Duy trì hội thoại, tích lũy kinh nghiệm qua nhiều phiên	Cửa sổ ngữ cảnh, Vector DB, Knowledge Graph
Công cụ & hành động	Thu thập dữ liệu thực tế và tác động vào hệ thống	Function Calling, API, truy vấn SQL, trình thông dịch mã

Chương 2

Sự tiến hóa mô hình: từ Chatbot đến Agentic Mesh

2.1 Đối sánh Non-agentic Chatbot, AI Copilot và Autonomous Agent

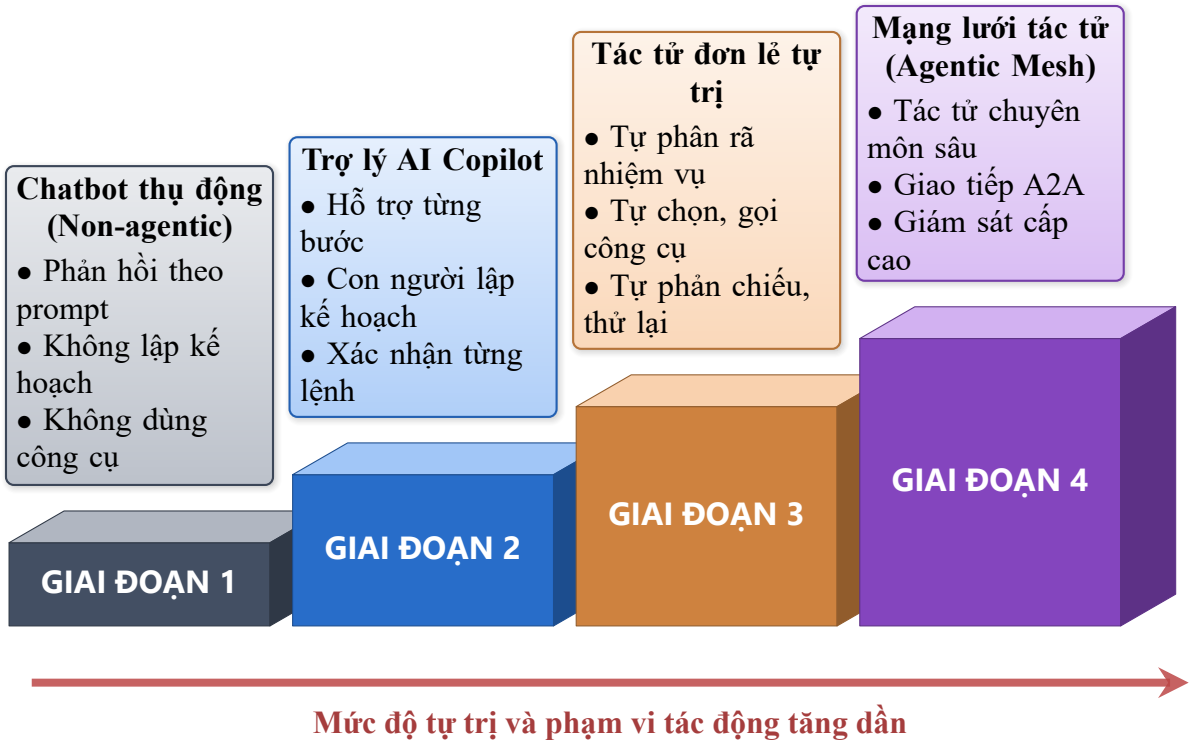
Để làm rõ khác biệt về bản chất kỹ thuật, Bảng 2.1 đối sánh ba thế hệ công nghệ theo các tiêu chí vận hành tổng hợp từ Google Cloud [1] và McKinsey [3]. Google Cloud phân biệt rõ *bot* (làm theo quy tắc định sẵn, ít tự chủ), *trợ lý AI* (hỗ trợ người dùng, người dùng vẫn ra quyết định) và *tác tử* (tự chủ cao nhất, tự đưa ra và thực hiện quyết định để đạt mục tiêu).

Bảng 2.1. Đối sánh Chatbot truyền thống, trợ lý AI Copilot và hệ tác tử tự trị.

Tiêu chí	Non-agentic Chatbot	AI Copilot (trợ lý)	Autonomous AI Agent
Mức độ tự trị	Thụ động, phụ thuộc hoàn toàn vào câu lệnh	Hỗ trợ từng bước, người dùng quyết định	Tự trị cao, tự điều hướng tới mục tiêu
Lập kế hoạch	Không có	Con người tự lập kế hoạch	Tự phân rã nhiệm vụ nhiều bước
Sử dụng công cụ	Không có hoặc rất hạn chế	Gợi ý, con người xác nhận từng lệnh	Tự lựa chọn và thực thi công cụ
Quản lý bộ nhớ	Ngữ cảnh phiên ngắn	Ngữ cảnh theo phiên làm việc	Bộ nhớ dài hạn, lưu vết trạng thái
Cơ chế sửa sai	Không tự sửa lỗi	Dựa vào phản hồi của người dùng	Tự phản chiếu (self-reflection), thử lại
Vai trò con người	Đặt câu hỏi, đọc kết quả	Thao tác chính, AI gợi ý	Giám sát cấp cao (supervisor / HITL)

2.2 Lộ trình bốn giai đoạn tiến hóa công nghệ trong tổ chức

Hình 2.1 mô tả lộ trình chuyển dịch của AI trong doanh nghiệp. **Giai đoạn 1** là chatbot thụ động trả lời câu hỏi thường gặp. **Giai đoạn 2** là trợ lý Copilot cài sẵn trong công cụ làm việc, tăng năng suất cá nhân. **Giai đoạn 3** là tác tử đơn lẻ tự trị, đảm nhận trọn một quy trình. **Giai đoạn 4** là mạng lưới nhiều tác tử chuyên biệt phối hợp với nhau trên quy mô toàn doanh nghiệp, McKinsey gọi là *Agentic Mesh*.



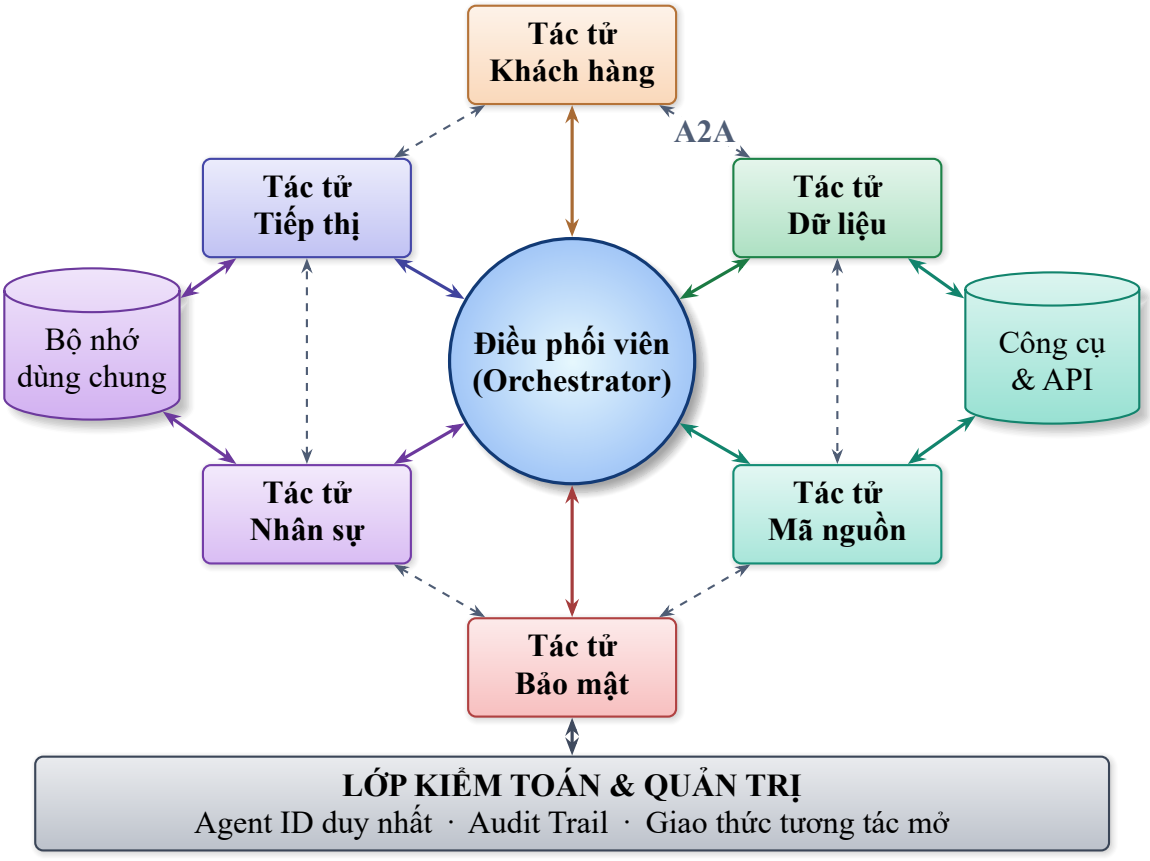
Hình 2.1. Lộ trình tiến hóa công nghệ từ Chatbot thụ động đến mạng lưới tác tử doanh nghiệp (Agentic Mesh).

Mỗi bước tiến kéo theo hai thay đổi: mức tự trị tăng và vai trò của con người dịch chuyển từ *người thao tác* sang *người giám sát*. Vì vậy các yêu cầu về kiểm soát, nhật ký và phân quyền (Chương 6) cũng tăng tương ứng.

2.3 Nguyên lý mạng lưới tác tử (Agentic Mesh) theo McKinsey

McKinsey [3] mô tả tương lai của tổ chức là một mạng lưới các tác tử chuyên môn hóa sâu, phối hợp qua các giao thức tương tác giữa tác tử với tác tử (Agent-to-Agent communication) để vận hành những quy trình nghiệp vụ phức hợp mà không cần con người can thiệp thủ công liên tục. Các đặc điểm chính của mô hình này:

- **Chuyên môn hóa.** Mỗi tác tử phụ trách một miền hẹp (tra cứu đơn hàng, phân tích dữ liệu, kiểm tra tuân thủ…), dễ kiểm thử và thay thế độc lập.
- **Điều phối.** Một tác tử điều phối (orchestrator) nhận mục tiêu, phân việc và tổng hợp kết quả.
- **Hạ tầng dùng chung.** Các tác tử chia sẻ bộ nhớ, kho công cụ và lớp kiểm toán chung để mọi hành động đều truy vết được.



Hình 2.2. Mạng lưới tác tử (Agentic Mesh): tác tử điều phối, các tác tử chuyên biệt liên lạc A2A, bộ nhớ – công cụ dùng chung và lớp kiểm toán.

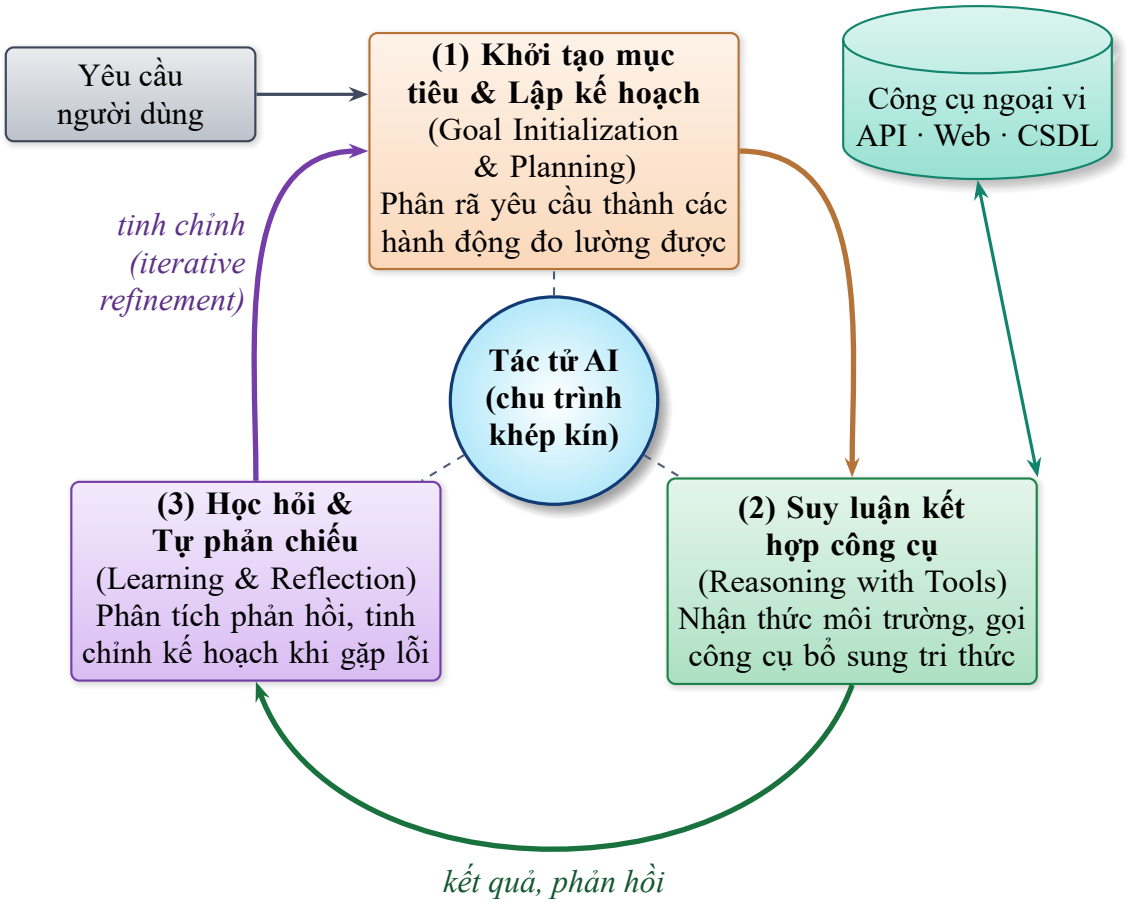
Chương 3

Kiến trúc kỹ thuật và cơ chế suy luận theo IBM Think

3.1 Chu trình nhận thức ba giai đoạn (IBM Cognitive Cycle)

IBM Think [2] mô tả hoạt động của một AI Agent qua ba giai đoạn khép kín (Hình 3.1).

1. **Khởi tạo mục tiêu và lập kế hoạch (Goal Initialization & Planning).** Con người hoặc hệ thống đặt mục tiêu và các ràng buộc; tác tử phân rã yêu cầu thành những tác vụ con có thể đo lường được. Với tác vụ đơn giản, bước lập kế hoạch có thể lược bỏ.
2. **Suy luận với công cụ sẵn có (Reasoning with Available Tools).** Tác tử không biết hết mọi thứ, nên nó dùng công cụ (tìm kiếm web, API, cơ sở dữ liệu, thậm chí các tác tử khác) để bổ sung thông tin còn thiếu, rồi cập nhật kế hoạch dựa trên dữ liệu mới.
3. **Học hỏi và phản chiếu (Learning & Reflection).** Tác tử tiếp nhận phản hồi, từ người dùng hoặc từ tác tử khác, lưu những gì đã học vào bộ nhớ và tinh chỉnh kế hoạch theo vòng lặp (iterative refinement) để không lặp lại lỗi cũ.



Hình 3.1. Chu trình nhận thức ba giai đoạn của AI Agent theo IBM Think.

3.2 Đối chiếu hai mô hình suy luận: ReAct và ReWOO

IBM Think [2] chỉ ra hai trường phái suy luận then chốt định hình hiệu năng của tác tử (Hình 3.2).

3.2.1 ReAct (Reasoning and Action)

ReAct [6] yêu cầu mô hình xen kẽ *suy nghĩ* và *hành động*: sau mỗi hành động, tác tử quan sát kết quả rồi mới suy nghĩ bước kế tiếp. Vòng lặp có dạng

$$\text{Thought} \rightarrow \text{Action} \rightarrow \text{Observation} \rightarrow \text{Thought} \rightarrow \cdots \rightarrow \text{Final Answer}.$$

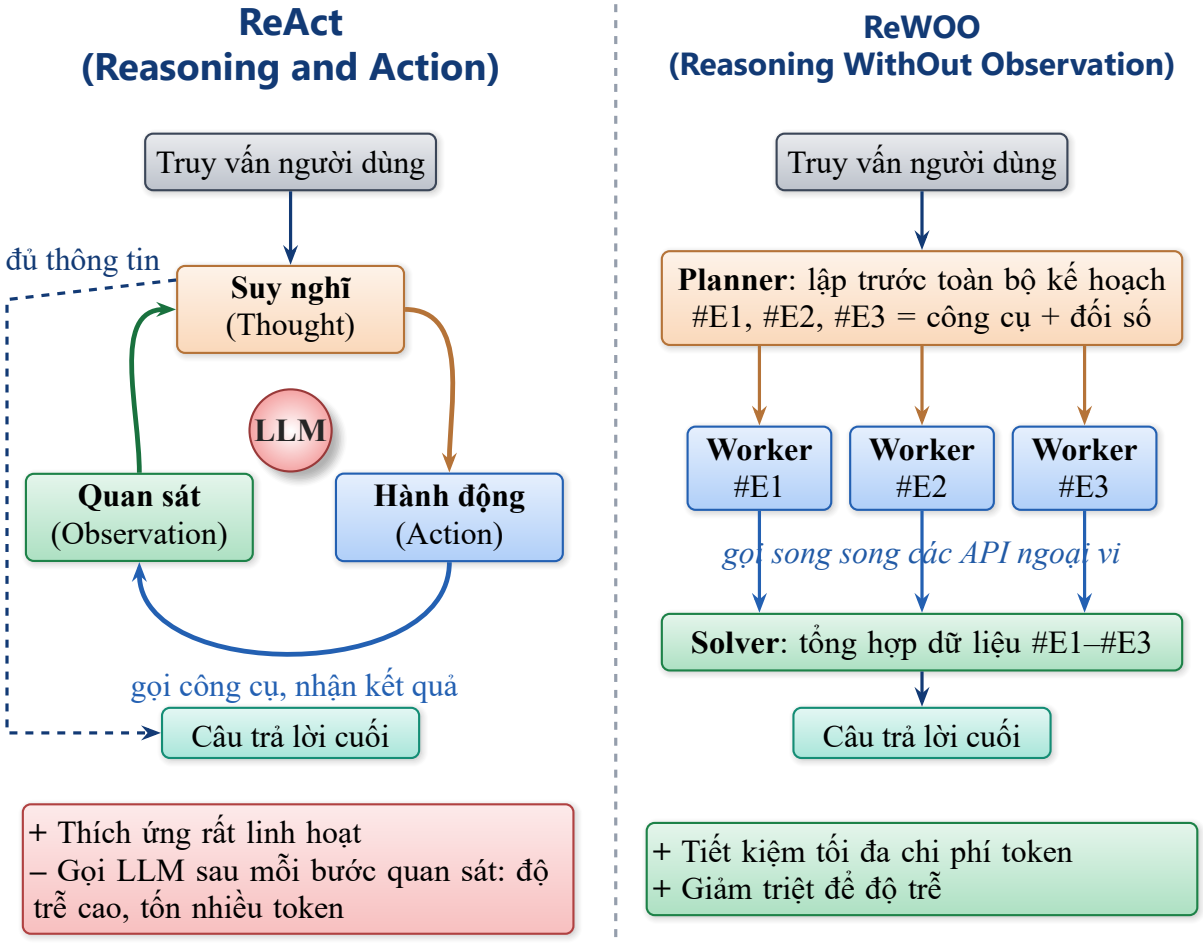
Ưu điểm là thích nghi linh hoạt: nếu một công cụ trả về lỗi, bước suy nghĩ tiếp theo sẽ thấy và đổi hướng. Nhược điểm là mỗi bước quan sát đòi hỏi một lần gọi LLM với toàn bộ ngữ cảnh tích lũy, nên độ trễ và chi phí token tăng theo số bước.

3.2.2 ReWOO (Reasoning Without Observation)

ReWOO [7] tách pha lập kế hoạch khỏi pha thực thi và gồm ba mô-đun:

- **Planner** lập trước toàn bộ kế hoạch, kể cả các lời gọi công cụ, dùng biến giữ chỗ (#E1, #E2,…) cho kết quả chưa có.
- **Worker** thực thi các lời gọi công cụ, có thể song song khi không phụ thuộc nhau.
- **Solver** tổng hợp kế hoạch và các bằng chứng thu được thành câu trả lời cuối.

Vì LLM chỉ được gọi ở pha lập kế hoạch và pha tổng hợp, ReWOO tiết kiệm token và giảm độ trễ. Đổi lại, kế hoạch đã lập không tự thay đổi khi kết quả trung gian bất ngờ.



Hình 3.2. So sánh cơ chế vận hành: ReAct (vòng lặp suy nghĩ – hành động – quan sát liên tục) và ReWOO (lập kế hoạch trước, thực thi song song, tổng hợp sau).

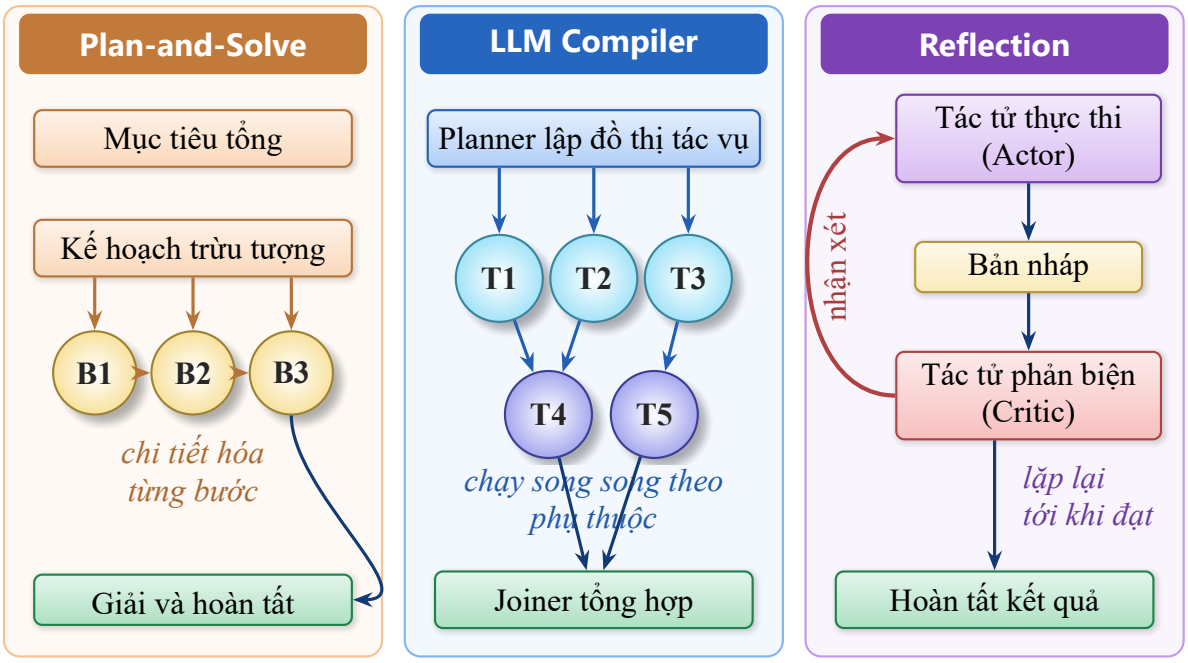
Bảng 3.1. Đối chiếu ReAct và ReWOO.

Tiêu chí	ReAct	ReWOO
Thứ tự suy luận – thực thi	Xen kẽ từng bước	Lập kế hoạch toàn bộ trước khi thực thi
Số lần gọi LLM	Một lần sau mỗi quan sát	Hai lần (Planner và Solver)
Khả năng thích nghi	Cao, đổi hướng ngay khi gặp kết quả bất ngờ	Thấp hơn, kế hoạch cố định
Chi phí token, độ trễ	Cao, tăng theo số bước	Thấp, công cụ có thể chạy song song
Phù hợp	Bài toán khám phá, môi trường biến động	Quy trình có cấu trúc, cần tiết kiệm chi phí

3.3 Các mô hình suy luận nâng cao khác

Ngoài ReAct và ReWOO, các tác tử hiện đại còn áp dụng thêm nhiều mẫu suy luận (Hình 3.3):

- **Plan-and-Solve:** lập kế hoạch trừu tượng ở mức cao trước, sau đó chi tiết hóa và giải từng bước.
- **LLM Compiler:** biểu diễn các tác vụ thành đồ thị phụ thuộc và thực thi song song những nhánh độc lập, giống trình biên dịch tối ưu luồng thực thi.
- **Reflection:** một tác tử (hoặc chính tác tử đó ở vai trò khác) đóng vai phản biện độc lập, rà soát kết quả và yêu cầu sửa trước khi hoàn tất.



Hình 3.3. Ba mẫu suy luận nâng cao: Plan-and-Solve, LLM Compiler (đồ thị tác vụ song song) và Reflection (tác tử thực thi – tác tử phản biện).

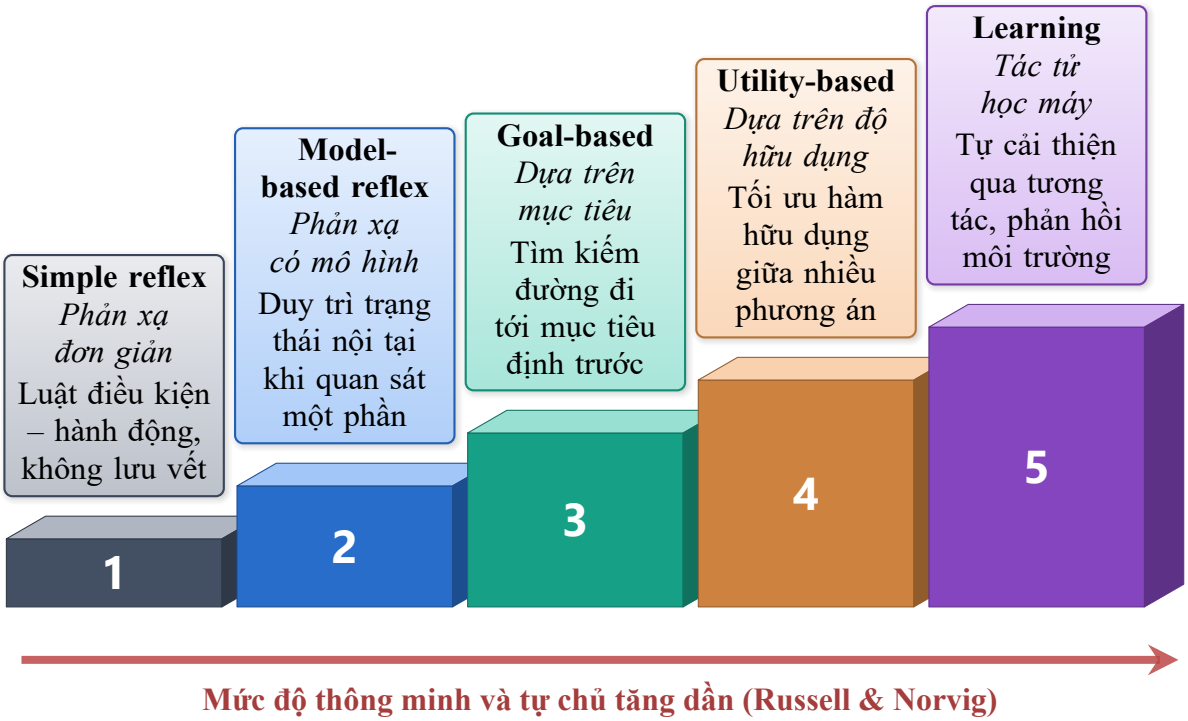
Chương 4

Năm phân cấp tác tử kinh điển và mô hình học máy

4.1 Năm loại tác tử theo Russell & Norvig

IBM Think [2] quy chiếu các kiến trúc tác tử hiện đại về năm loại tác tử kinh điển của Russell & Norvig [5], xếp theo mức độ phức tạp tăng dần (Hình 4.1).

1. **Simple reflex agent** (tác tử phản xạ đơn giản): hành động theo luật *điều kiện – hành động* dựa trên tri giác hiện tại, không lưu vết. Ví dụ: bộ điều nhiệt bật máy sưởi khi nhiệt độ dưới ngưỡng.
2. **Model-based reflex agent** (tác tử phản xạ có mô hình): duy trì một mô hình trạng thái nội tại để xử lý môi trường chỉ quan sát được một phần. Ví dụ: robot hút bụi nhớ những vùng đã dọn.
3. **Goal-based agent** (tác tử dựa trên mục tiêu): ra quyết định dựa trên mục tiêu định trước, tìm kiếm và lập kế hoạch chuỗi hành động dẫn tới mục tiêu. Ví dụ: hệ thống chỉ đường tìm lộ trình tới địa điểm.
4. **Utility-based agent** (tác tử dựa trên hữu dụng): khi có nhiều cách đạt mục tiêu, chọn cách tối đa hóa hàm hữu dụng (thời gian, chi phí, an toàn). Ví dụ: chỉ đường cân bằng thời gian và phí cầu đường.
5. **Learning agent** (tác tử học máy): tự cải thiện năng lực qua tương tác và phản hồi của môi trường.



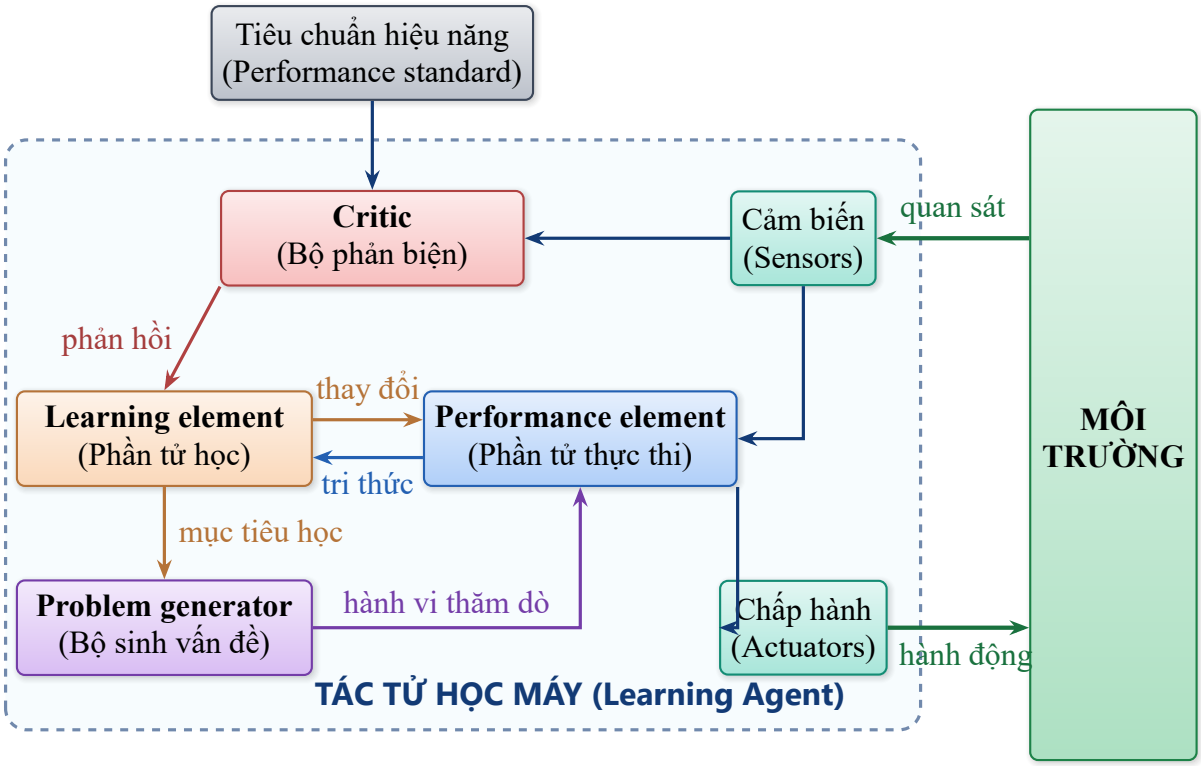
Hình 4.1. Năm loại tác tử kinh điển của Russell & Norvig xếp theo mức tự chủ và năng lực tăng dần.

4.2 Cấu trúc nội tại của tác tử học máy (Learning Agent)

Theo Russell & Norvig [5], một Learning Agent gồm bốn thành phần chức năng (Hình 4.2):

- **Critic** (bộ phê bình): đối chiếu hành vi với *tiêu chuẩn hiệu năng* cố định bên ngoài và sinh tín hiệu phản hồi cho biết tác tử đang làm tốt hay kém.
- **Learning Element** (phần tử học): nhận phản hồi từ Critic để cập nhật tri thức, tức chính sách của tác tử.
- **Performance Element** (phần tử thực thi): dựa trên tri thức hiện có, chọn hành động đưa vào môi trường.
- **Problem Generator** (bộ sinh vấn đề): đề xuất những hành động thăm dò mới để tác tử tích lũy kinh nghiệm, kể cả khi trước mắt chúng chưa phải lựa chọn tốt nhất.

Bốn thành phần này ứng với các phần của tác tử Q-Learning xây dựng ở Chương 7 (Bảng 4.1).



Hình 4.2. Kiến trúc tổng quát của tác tử học máy (Learning Agent Architecture) theo Russell & Norvig.

Bảng 4.1. Bốn thành phần của Learning Agent trong tác tử Q-Learning của bài thực hành.

Thành phần	**Hiện thực trong tác tử Q-Learning**
Tiêu chuẩn hiệu năng	Hàm phần thưởng: -1 mỗi bước, -10 khi đâm tường lửa, $+100$ khi về đích
Critic	Sai số TD $\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$
Learning Element	Phép cập nhật $Q(s, a) \leftarrow Q(s, a) + \alpha\, \delta$ (phương thức `update`)
Performance Element	Chọn $\arg\max_a Q(s, a)$ (nhánh khai thác của `choose_action`)
Problem Generator	Nhánh thăm dò ngẫu nhiên với xác suất ε

4.3 Khung toán học Quá trình Quyết định Markov (MDP)

Mọi bài toán tác tử học tương tác với môi trường đều được mô hình hóa bằng **Quá trình Quyết định Markov** (Markov Decision Process) [8], gồm bộ năm thành phần

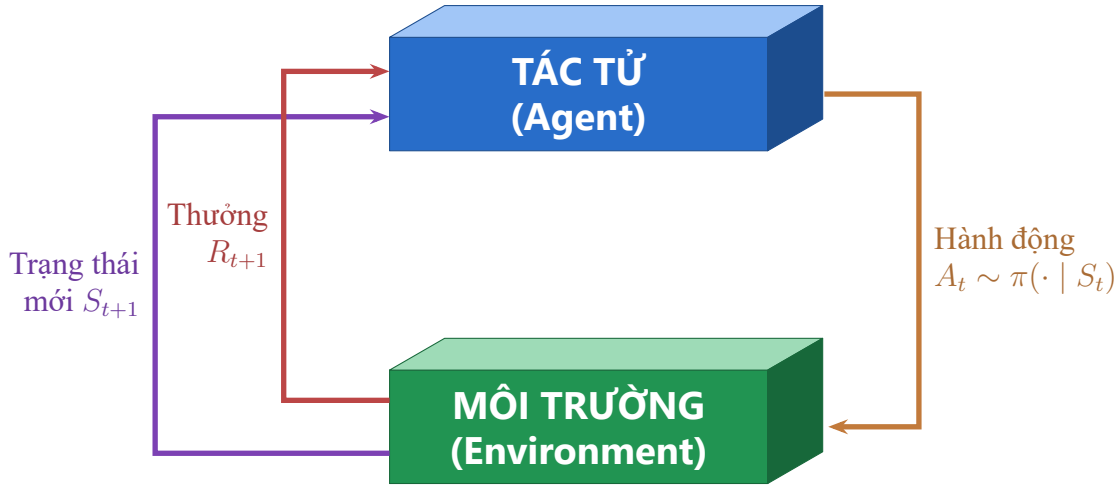
$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle,$$

trong đó:

- $\mathcal{S}$ – **không gian trạng thái**, mô tả những gì tác tử nhận thức được về môi trường;
- $\mathcal{A}$ – **không gian hành động**, tập các hành vi hợp lệ;
- $P(s' \mid s, a)$ – **hàm chuyển trạng thái**, xác suất sang trạng thái s' khi thực hiện a tại s;
- $R(s, a, s')$ – **hàm phần thưởng**, đại lượng vô hướng tức thời phản hồi chất lượng hành vi;
- $\gamma \in [0, 1]$ – **hệ số chiết khấu**, cân bằng giữa lợi ích tức thời và mục tiêu dài hạn.

Tính chất Markov phát biểu rằng tương lai chỉ phụ thuộc trạng thái hiện tại, không phụ thuộc quá khứ: $P(S_{t+1} \mid S_t, A_t, S_{t-1}, \dots) = P(S_{t+1} \mid S_t, A_t)$. Mục tiêu của tác tử là tìm chính sách π tối đa hóa tổng phần thưởng chiết khấu kỳ vọng

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$



Quá trình Quyết định Markov: $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$

$\mathcal{S}$: không gian trạng thái (nhận thức môi trường)

$\mathcal{A}$: không gian hành động hợp lệ

$P(s' \mid s, a)$: xác suất chuyển trạng thái

$R(s, a, s')$: phần thưởng tức thời, vô hướng

$\gamma \in [0, 1]$: hệ số chiết khấu, cân bằng lợi ích tức thời và mục tiêu dài hạn

Hình 4.3. Vòng tương tác tác tử – môi trường của một Quá trình Quyết định Markov.

Chương 5

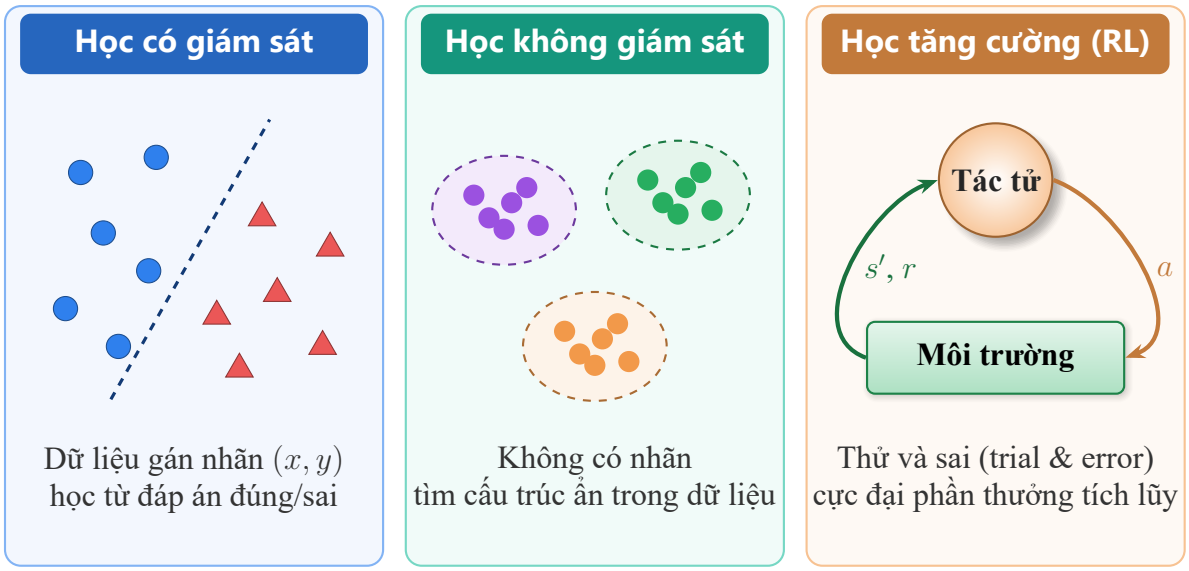
Lý thuyết học tăng cường theo GeeksforGeeks

5.1 Bản chất của học tăng cường (Reinforcement Learning – RL)

Theo GeeksforGeeks [4], học tăng cường là một nhánh của học máy, trong đó một tác tử tự trị học cách đưa ra chuỗi quyết định tối ưu bằng cách tương tác với môi trường để tối đa hóa tổng phần thưởng tích lũy. Tác tử không được chỉ dẫn hành động nào là đúng; nó học qua **thử và sai** (trial and error), nhận phần thưởng khi làm tốt và hình phạt khi làm sai. Các thành phần cốt lõi gồm: **tác tử** (agent), **môi trường** (environment), **trạng thái**, **hành động**, **phần thưởng**, **chính sách** (policy) và **hàm giá trị** (value function).

Điểm khác biệt với hai mô thức học máy còn lại (Hình 5.1):

- **So với học có giám sát (Supervised):** không có tập dữ liệu gán nhãn đúng/sai từ trước; tác tử phải tự khám phá hành vi tối ưu qua tín hiệu thưởng/phạt, và tín hiệu này có thể đến trễ nhiều bước.
- **So với học không giám sát (Unsupervised):** không chỉ tìm cấu trúc ẩn trong dữ liệu mà hướng tới cực đại hóa một hàm mục tiêu số học là tổng phần thưởng.



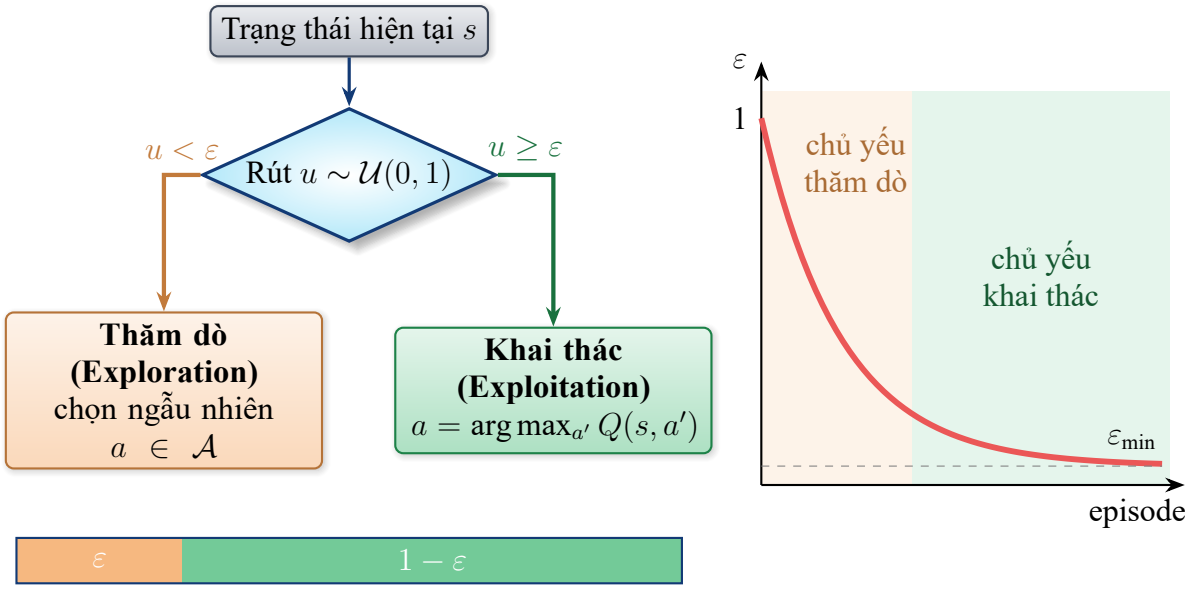
Hình 5.1. Ba mô thức học máy: học có giám sát, học không giám sát và học tăng cường.

5.2 Bài toán đánh đổi thăm dò (Exploration) và khai thác (Exploitation)

Tác tử luôn đứng trước một thế khó: *khai thác* hành động đã biết để nhận phần thưởng chắc chắn, hay *thăm dò* hành động mới để có thể tìm ra phần thưởng lớn hơn. Chỉ khai thác thì dễ mắc kẹt ở lời giải chưa tối ưu; chỉ thăm dò thì không bao giờ tận dụng được điều đã học. Chiến lược phổ biến nhất là ε-**greedy**:

$$a = \begin{cases} \text{hành động ngẫu nhiên trong } \mathcal{A}, & \text{với xác suất } \varepsilon \quad \text{(thăm dò)}, \\ \arg\max_{a'} Q(s, a'), & \text{với xác suất } 1 - \varepsilon \quad \text{(khai thác)}. \end{cases}$$

Trong thực tế ε được giảm dần theo thời gian: đầu quá trình tác tử chưa biết gì nên cần thăm dò nhiều; khi Q-Table đã đáng tin, tác tử chuyển sang khai thác. Bài thực hành dùng lịch giảm theo cấp số nhân $\varepsilon_{k+1} = \max(\varepsilon_{\min}, \varepsilon_k \cdot \lambda)$ với $\varepsilon_0 = 1$, $\varepsilon_{\min} = 0{,}01$ và $\lambda = 0{,}998$.



Hình 5.2. Chính sách ε-greedy: phân nhánh thăm dò – khai thác và lịch giảm dần của ε.

5.3 Hàm giá trị và phương trình tối ưu Bellman

Hàm giá trị hành động $Q^\pi(s,a)$ biểu thị kỳ vọng tổng phần thưởng chiết khấu khi tác tử thực hiện hành động a tại trạng thái s rồi sau đó tuân theo chính sách π:

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]\,.$$

Hàm giá trị tối ưu Q^* thỏa **phương trình tối ưu Bellman** [8]:

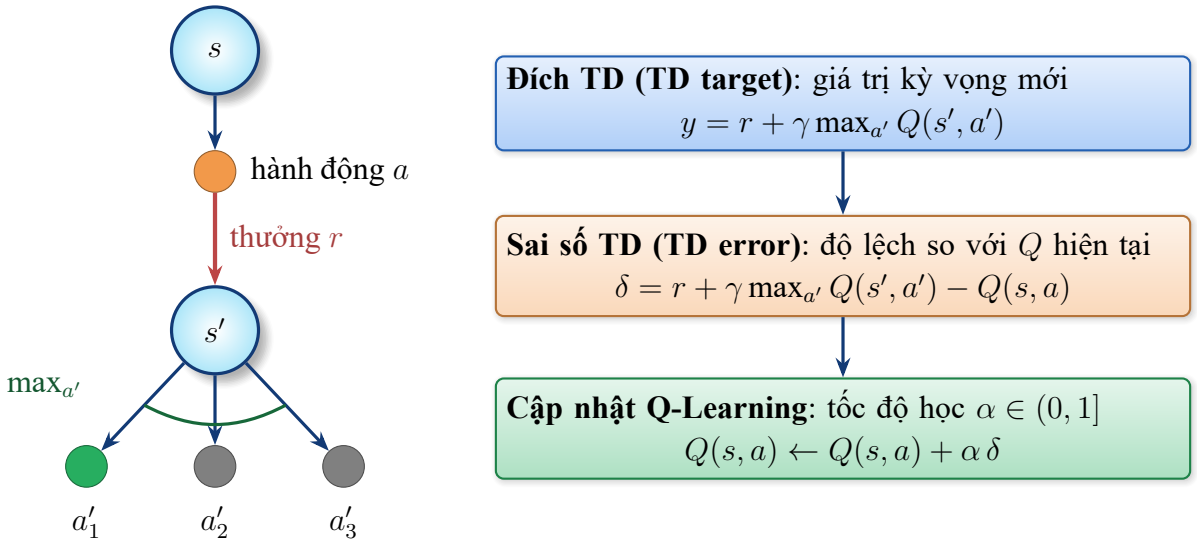
$$Q^*(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \;\Big|\; S_t = s, A_t = a\Big]\,.$$

Thuật toán **Q-Learning** xấp xỉ Q^* bằng cách lấy mẫu từ kinh nghiệm thực tế và cập nhật dần theo quy tắc

$$Q(s,a) \;\leftarrow\; Q(s,a) + \alpha\Big[\underbrace{R + \gamma \max_{a'} Q(s',a')}_{\text{mục tiêu TD}} - Q(s,a)\Big],$$

trong đó $\alpha \in (0,1]$ là tốc độ học và $\gamma \in [0,1]$ là hệ số chiết khấu. Phần trong ngoặc vuông là *sai số TD* (temporal difference). Khi tác tử về trạng thái kết thúc, mục tiêu TD chỉ còn là R vì không còn phần thưởng tương lai. Q-Learning là thuật toán *off-policy*: nó học chính sách tham lam tối ưu trong khi hành động

theo ε-greedy.



Hình 5.3. Sơ đồ sao lưu (backup diagram) của phép cập nhật Q-Learning.

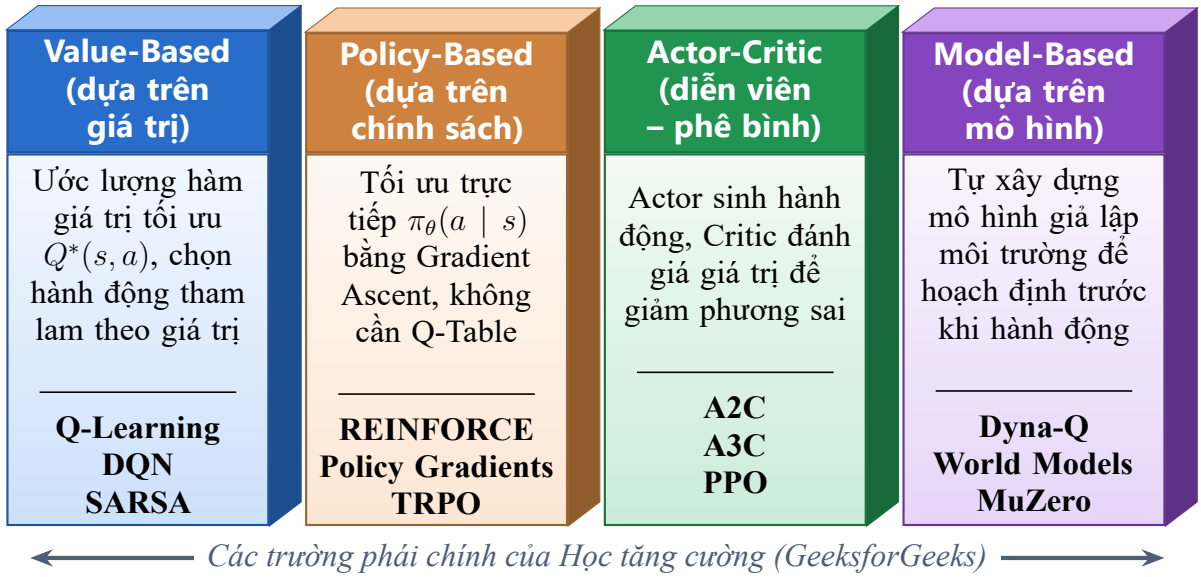
5.4 Phân loại các trường phái học tăng cường

Bảng 5.1 và Hình 5.4 tóm tắt bốn trường phái chính.

Bảng 5.1. Phân loại các trường phái học tăng cường chính.

Trường phái	Đặc trưng kỹ thuật cốt lõi	Thuật toán tiêu biểu
Value-Based	Ước lượng hàm giá trị tối ưu $Q^*(s, a)$, chọn hành động tham lam theo giá trị	Q-Learning, SARSA, Deep Q-Network (DQN)
Policy-Based	Tối ưu trực tiếp chính sách $\pi_\theta(a \mid s)$ bằng gradient ascent, không cần Q-Table	REINFORCE, Policy Gradient, TRPO
Actor-Critic	Kết hợp cả hai: Actor sinh hành động, Critic đánh giá giá trị để giảm phương sai	A2C, A3C, PPO
Model-Based	Tự xây dựng mô hình giả lập môi trường để lập kế hoạch trước khi hành động	Dyna-Q, World Models, MuZero

GeeksforGeeks [4] cũng nêu các ứng dụng điển hình của học tăng cường: robot và điều khiển tự động, trò chơi (cờ vây, game điện tử), xe tự lái, hệ gợi ý, tài chính định lượng, và tinh chỉnh mô hình ngôn ngữ theo phản hồi con người (Chương 9).



Hình 5.4. Bốn trường phái học tăng cường và các thuật toán tiêu biểu.

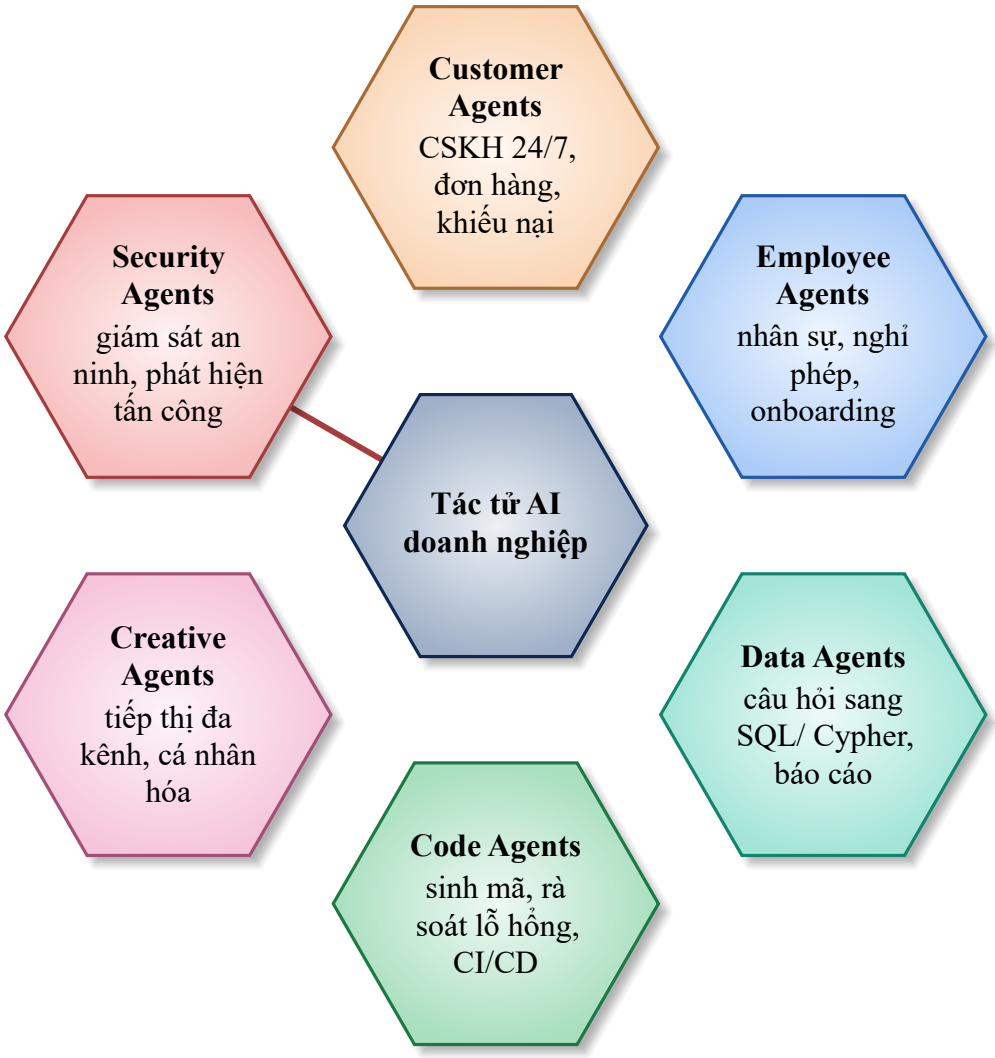
Chương 6

Taxonomy ứng dụng doanh nghiệp, rủi ro và quản trị an toàn

6.1 Phân loại tác tử doanh nghiệp theo Google Cloud và McKinsey

Google Cloud [1] và McKinsey [3] nhận diện sáu nhóm tác tử chủ lực trong doanh nghiệp (Hình 6.1):

- **Customer Agents:** chăm sóc khách hàng 24/7, tra cứu đơn hàng, hỗ trợ khiếu nại và đổi trả theo quy chế.
- **Employee Agents:** hỗ trợ nhân sự nội bộ, giải đáp quy trình, đăng ký nghỉ phép, đón nhân viên mới (onboarding).
- **Data Agents:** phân tích dữ liệu kinh doanh, chuyển câu hỏi ngôn ngữ tự nhiên thành truy vấn SQL/Cypher và xuất báo cáo tự động.
- **Code Agents:** sinh mã nguồn, rà soát lỗ hổng bảo mật, viết kiểm thử đơn vị, hỗ trợ CI/CD.
- **Creative Agents:** thiết kế thông điệp tiếp thị đa kênh, cá nhân hóa nội dung theo hồ sơ khách hàng.
- **Security Agents:** giám sát nhật ký an ninh mạng, phát hiện tấn công bất thường và kích hoạt quy trình ứng phó.



Hình 6.1. Sáu nhóm tác tử doanh nghiệp theo Google Cloud và McKinsey.

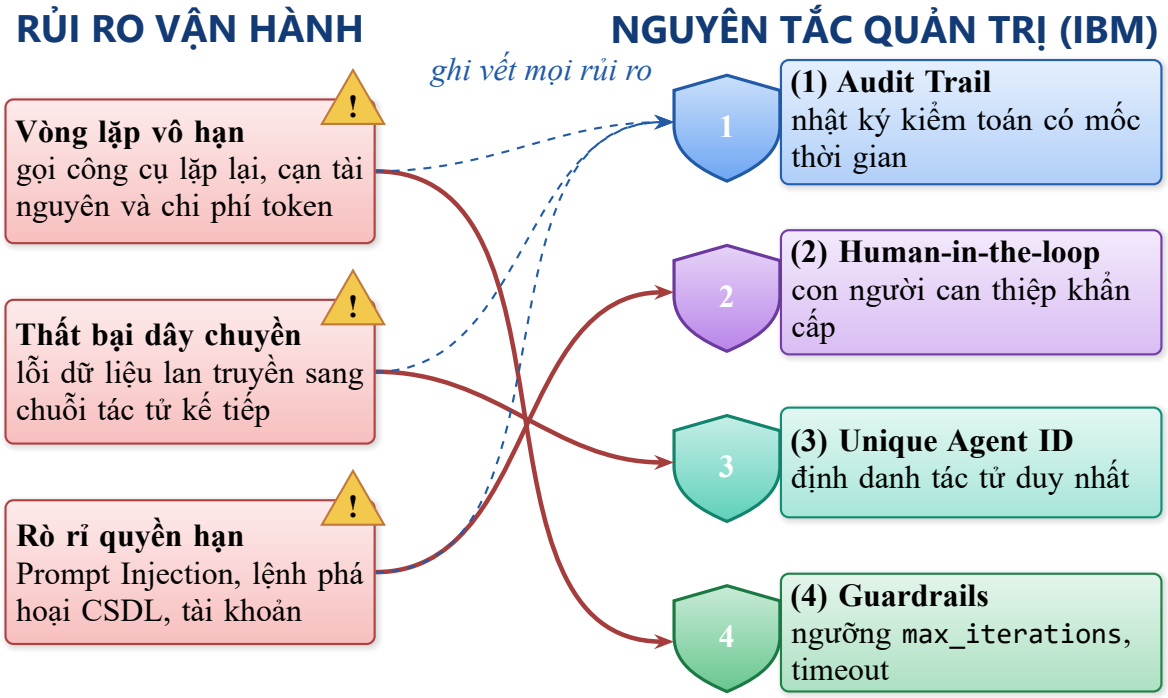
6.2 Thách thức kỹ thuật và bốn nguyên tắc quản trị theo IBM Think

Bên cạnh lợi ích, IBM Think [2] cảnh báo ba thách thức lớn:

- **Vòng lặp vô hạn (Infinite Loops):** tác tử gọi công cụ lặp đi lặp lại khi không lập được kế hoạch hoàn chỉnh hoặc không đánh giá được kết quả.
- **Thất bại lan truyền (Cascading Failures):** trong hệ đa tác tử dựng trên cùng một mô hình nền tảng, một điểm yếu chung hoặc dữ liệu sai từ một tác tử có thể kéo theo cả chuỗi.
- **Rò rỉ đặc quyền (Privilege Escalation):** tác tử được cấp quyền truy cập hệ thống có thể bị lợi dụng, ví dụ bằng tấn công chèn lệnh (prompt injection), để thực hiện thao tác vượt thẩm quyền.

Để bảo đảm an toàn, IBM đề xuất bốn nguyên tắc quản trị:

1. **Audit Trail** – nhật ký hoạt động có dấu thời gian cho mọi lời gọi công cụ và quyết định, để truy vết khi có sự cố.
2. **HITL (Human-in-the-loop)** – con người có thể can thiệp, phê duyệt hoặc dừng khẩn cấp, đặc biệt với thao tác có hệ quả lớn.
3. **Unique Agent IDs** – mỗi tác tử có định danh duy nhất để biết hành động nào do tác tử nào thực hiện.
4. **Guardrails** – ngưỡng giới hạn cứng như số vòng lặp tối đa (`max_iterations`), thời gian chờ, cơ chế tự ngắt.



Hình 6.2. Ba nhóm rủi ro vận hành và bốn nguyên tắc quản trị tương ứng.

Tác tử Q-Learning của bài thực hành cũng có một guardrail theo đúng tinh thần này: hằng số `MAX_STEPS` giới hạn số bước tối đa của mỗi episode, nên ở giai đoạn đầu, khi tác tử còn đi ngẫu nhiên, nó không thể lặp vô hạn trong mê cung.

Bảng 6.1. Ma trận kiểm soát rủi ro vận hành AI Agent theo khuyến nghị của IBM và Google Cloud.

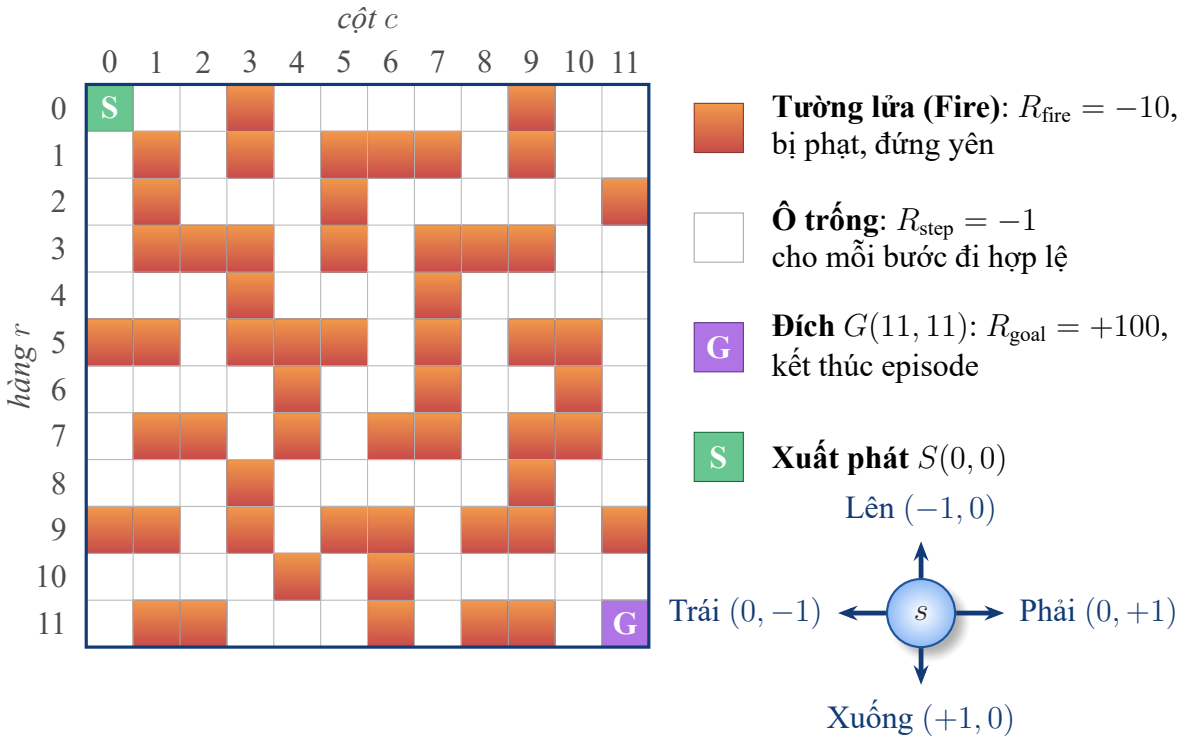
Nhóm rủi ro	**Biểu hiện và hậu quả**	**Cơ chế kiểm soát**
Vòng lặp vô hạn	Tác tử gọi công cụ lặp lại khi gặp lỗi, cạn kiệt tài nguyên máy chủ và chi phí token	Ngưỡng cứng `max_iterations`, giới hạn thời gian, tự ngắt
Thất bại dây chuyền	Dữ liệu sai từ một tác tử lan sang chuỗi tác tử kế tiếp trong mạng mesh	Định danh tác tử, kiểm tra dữ liệu trung gian, cô lập lỗi cục bộ
Rò rỉ quyền hạn	Tác tử bị chèn lệnh, thực hiện thao tác phá hoại dữ liệu hoặc tài khoản thanh toán	HITL cho thao tác quan trọng, quyền tối thiểu (least privilege)

Chương 7

Thực nghiệm lập trình tác tử học tăng cường trên Jupyter Notebook

7.1 Mô tả bài toán điều hướng mê cung

Để hiện thực hóa cơ chế tự học của Learning Agent (Chương 4) bằng thuật toán Q-Learning (Chương 5), bài thực hành xây dựng chương trình hoàn chỉnh trên Jupyter Notebook `agent01_rl_qlearning.ipynb` giải bài toán **robot điều hướng mê cung** theo tài liệu GeeksforGeeks [4]. Mê cung do sinh viên tự thiết kế, kích thước 12×12 với 55 ô tường lửa (Fire). Xuất phát từ ô $S\,(0,0)$, robot không biết trước bản đồ; nó phải tự khám phá, học cách tránh tường lửa và tìm ra lộ trình ngắn nhất về đích $G\,(11,11)$ với tổng phần thưởng lớn nhất (Hình 7.1).



Hình 7.1. Mê cung 12×12 của bài thực hành: ô xuất phát S, ô đích G, các ô tường lửa, bốn hành động và hàm phần thưởng.

7.1.1 Bốn thành tố của bài toán học tăng cường

Bài toán được mô hình hóa thành một MDP (Bảng 7.1):

1. **Trạng thái** $\mathcal{S}$: tọa độ ô (r, c) với $0 \le r, c \le 11$, tổng cộng 144 trạng thái lưới. Mỗi trạng thái được đánh chỉ số $r \cdot 12 + c$ để làm hàng của Q-Table.
2. **Hành động** $\mathcal{A}$: bốn hướng rời rạc Trái $(0, -1)$, Phải $(0, 1)$, Lên $(-1, 0)$, Xuống $(1, 0)$.
3. **Phần thưởng** R: $R_{\text{fire}} = -10$ khi đâm vào tường lửa hoặc ra khỏi mê cung (robot đứng yên tại chỗ), $R_{\text{goal}} = +100$ khi về đích (kết thúc episode), $R_{\text{step}} = -1$ cho mỗi bước đi hợp lệ để thúc tác tử đi đường ngắn.
4. **Quy tắc học**: cập nhật Bellman $Q(s,a) \leftarrow Q(s,a) + \alpha\,[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ với $\alpha = 0{,}1$, $\gamma = 0{,}95$ và lịch ε-greedy giảm từ 1 về 0,01 (nhân 0,998 sau mỗi episode), huấn luyện 3.000 episode, tối đa 300 bước mỗi episode.

Bảng 7.1. Các thành phần MDP của bài toán mê cung.

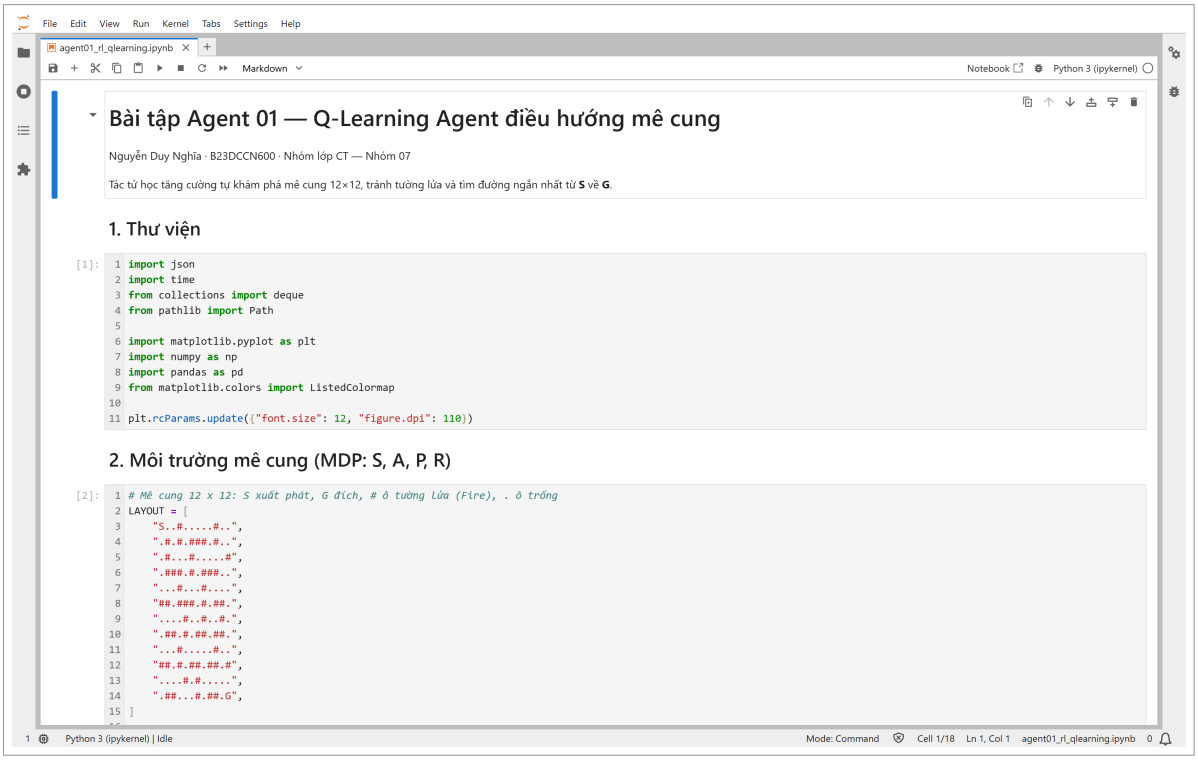
Thành phần	Giá trị trong bài thực hành
$\mathcal{S}$	144 ô (r, c); robot chỉ đứng được trên ô trống
$\mathcal{A}$	{Trái, Phải, Lên, Xuống}
$P(s' \mid s, a)$	Tất định: sang ô kề theo hướng a; nếu ô đó là tường lửa hoặc ngoài biên thì $s' = s$
$R(s, a, s')$	$+100$ về đích, -10 đâm tường/biên, -1 bước thường
γ	0,95
Trạng thái kết thúc	Ô đích $G\,(11, 11)$, hoặc hết 300 bước

7.2 Môi trường lập trình

Notebook chạy trên JupyterLab 4 với Python 3.13, dùng NumPy cho Q-Table, pandas để hiển thị bảng và Matplotlib để vẽ. Q-Table chỉ có 144×4 ô số thực nên toàn bộ việc huấn luyện chạy trên CPU và mất 2,393 giây cho 3.000 episode; bài toán dạng bảng này không cần GPU. Notebook được tổ chức thành tám phần (Hình 7.2):

1. Thư viện; 2. Môi trường mê cung; 3. Q-Learning Agent và vòng huấn luyện; 4. Huấn luyện;

5. Lộ trình tối ưu theo chính sách tham lam; 6. Trực quan hóa; 7. Trích Q-Table; 8. Lưu số liệu.



Hình 7.2. Ảnh chụp màn hình JupyterLab mở notebook Q-Learning của bài tập: phần đầu notebook và ô mã thư viện.

7.3 Ảnh chụp màn hình mã nguồn thuật toán (Copy Screen Code)

Các hình dưới đây là ảnh chụp màn hình thực tế những khối mã xây dựng môi trường, thiết lập tham số và thuật toán Q-Learning Agent trên Jupyter Notebook, đúng yêu cầu của bài tập (copy screen code). Sau mỗi ảnh chụp, báo cáo in lại đoạn mã tương ứng ở dạng văn bản (trích nguyên văn từ mã đã chạy) kèm giải thích.

7.3.1 Khối thư viện

[1]:
```
import json
import time
from collections import deque
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.colors import ListedColormap

plt.rcParams.update({"font.size": 12, "figure.dpi": 110})
```

Hình 7.3. Ảnh chụp màn hình ô mã 1: nạp thư viện.

[2]:
```
# Mê cung 12 x 12: S xuất phát, G đích, # ô tường lửa (Fire), . ô trống
LAYOUT = [
    "S..#.....#..",
    ".#.#.###.#..",
    ".#...#.....#",
    ".###.#.###..",
    "...#...#....",
    "##.###.#.##.",
    "....#..#..#.",
    ".##.#.##.##.",
    "...#.....#..",
    "##.#.##.##.#",
    "....#.#.....",
    ".##...#.##.G",
]

# Bốn hành động rời rạc: (tên, dịch chuyển hàng, dịch chuyển cột)
ACTIONS = [("Trái", 0, -1), ("Phải", 0, 1), ("Lên", -1, 0), ("Xuống", 1, 0)]

R_FIRE = -10.0   # đâm vào tường lửa hoặc ra khỏi mê cung: bị phạt và đứng yên
R_GOAL = +100.0  # về đích: kết thúc episode
R_STEP = -1.0    # mỗi bước đi hợp lệ: phạt nhẹ để tác tử chọn đường ngắn


class MazeEnv:
    """Môi trường mê cung lưới: trạng thái là tọa độ (r, c), 4 hành động, phần thưởng theo ô đích đến."""

    def __init__(self, layout=LAYOUT):
        self.grid = np.array([list(row) for row in layout])
        self.n_rows, self.n_cols = self.grid.shape
        self.start = tuple(int(v) for v in np.argwhere(self.grid == "S")[0])
        self.goal = tuple(int(v) for v in np.argwhere(self.grid == "G")[0])
        self.n_states = self.n_rows * self.n_cols
        self.n_actions = len(ACTIONS)
        self.state = self.start
```

Hình 7.4. Ảnh chụp màn hình ô mã 2 (phần 1/2): bản đồ mê cung, hành động, phần thưởng và đầu lớp `MazeEnv`.

7.3.2 Khối môi trường mê cung

Bản đồ được khai báo bằng các chuỗi ký tự để dễ đọc và dễ sửa: `S` là xuất phát, `G` là đích, `#` là tường lửa.

Lớp `MazeEnv` cài đặt hàm chuyển trạng thái P và hàm phần thưởng R của MDP. Phương thức `step` nhận một hành động, tính ô đích đến; nếu ô đó ra khỏi mê cung hoặc là tường lửa thì robot đứng yên và nhận R_{fire}; nếu là ô đích thì episode kết thúc với R_{goal}; còn lại nhận R_{step}. Phương thức `shortest_path_length` chạy

```python

    def is_fire(self, r, c):
        return self.grid[r, c] == "#"

    def to_index(self, state):
        return state[0] * self.n_cols + state[1]

    def reset(self):
        self.state = self.start
        return self.state

    def step(self, action):
        """Thực hiện hành động; trả về (trạng thái mới, phần thưởng, kết thúc?)."""
        _, dr, dc = ACTIONS[action]
        r, c = self.state[0] + dr, self.state[1] + dc
        if not (0 <= r < self.n_rows and 0 <= c < self.n_cols) or self.is_fire(r, c):
            return self.state, R_FIRE, False
        self.state = (r, c)
        if self.state == self.goal:
            return self.state, R_GOAL, True
        return self.state, R_STEP, False

    def shortest_path_length(self):
        """BFS trên mê cung (tác tử không biết hàm này) để kiểm chứng lộ trình học được có ngắn nhất không."""
        dist = {self.start: 0}
        queue = deque([self.start])
        while queue:
            r, c = queue.popleft()
            for _, dr, dc in ACTIONS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < self.n_rows and 0 <= nc < self.n_cols and not self.is_fire(nr, nc) \
                        and (nr, nc) not in dist:
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
        return dist.get(self.goal)
```

Hình 7.5. Ảnh chụp màn hình ô mã 2 (phần 2/2): phương thức `step` và BFS kiểm chứng của lớp `MazeEnv`.

Đoạn mã 7.1. Bản đồ mê cung 12×12.

```python
# ag1/maze.py
LAYOUT = [
    "S..#.....#..",
    ".#.#.###.#..",
    ".#...#.....#",
    ".###.#.###..",
    "...#...#....",
    "##.###.#.##.",
    "....#..#..#.",
    ".##.#.##.##.",
    "...#.....#..",
    "##.#.##.##.#",
    "....#.#.....",
    ".##...#.##.G",
]
```

BFS trên bản đồ. Tác tử *không* dùng hàm này; nó chỉ để kiểm chứng sau huấn luyện rằng lộ trình học được có đúng là ngắn nhất.

Đoạn mã 7.2. Lớp môi trường `MazeEnv`: trạng thái, bước chuyển, phần thưởng và BFS kiểm chứng.

```python
# ag1/maze.py
class MazeEnv:
    """Môi trường mê cung lưới: trạng thái là tọa độ (r, c), 4 hành động, phần thưởng theo ô đích đến."""

    def __init__(self, layout=LAYOUT):
        self.grid = np.array([list(row) for row in layout])
        self.n_rows, self.n_cols = self.grid.shape
        self.start = tuple(int(v) for v in np.argwhere(self.grid == "S")[0])
        self.goal = tuple(int(v) for v in np.argwhere(self.grid == "G")[0])
        self.n_states = self.n_rows * self.n_cols
        self.n_actions = len(ACTIONS)
        self.state = self.start

    def is_fire(self, r, c):
        return self.grid[r, c] == "#"

    def to_index(self, state):
        return state[0] * self.n_cols + state[1]

    def reset(self):
        self.state = self.start
        return self.state

    def step(self, action):
        """Thực hiện hành động; trả về (trạng thái mới, phần thưởng, kết thúc?)."""
        _, dr, dc = ACTIONS[action]
        r, c = self.state[0] + dr, self.state[1] + dc
        if not (0 <= r < self.n_rows and 0 <= c < self.n_cols) or self.is_fire(r, c):
            return self.state, R_FIRE, False
        self.state = (r, c)
        if self.state == self.goal:
            return self.state, R_GOAL, True
        return self.state, R_STEP, False

    def shortest_path_length(self):
        """BFS trên mê cung (tác tử không biết hàm này) để kiểm chứng lộ trình học được có ngắn nhất không."""
        dist = {self.start: 0}
```

```
        queue = deque([self.start])
        while queue:
            r, c = queue.popleft()
            for _, dr, dc in ACTIONS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < self.n_rows and 0 <= nc < self.n_cols and not
                ↪ self.is_fire(nr, nc) \
                        and (nr, nc) not in dist:
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
        return dist.get(self.goal)
```

7.3.3 Khối Q-Learning Agent và vòng huấn luyện

[3]:

```
# Siêu tham số của Q-Learning Agent
ALPHA = 0.1          # tốc độ học
GAMMA = 0.95         # hệ số chiết khấu
EPS_START = 1.0      # epsilon ban đầu: thăm dò hoàn toàn
EPS_MIN = 0.01       # epsilon tối thiểu
EPS_DECAY = 0.998    # epsilon nhân với hệ số này sau mỗi episode
EPISODES = 3000
MAX_STEPS = 300      # giới hạn bước mỗi episode (guardrail chống vòng lặp vô hạn)
LOG_EVERY = 250
SEED = 42


class QLearningAgent:
    """Learning Agent dạng bảng: Q-Table là tri thức, epsilon-greedy là Problem Generator,
    cập nhật Bellman là Learning Element."""

    def __init__(self, n_states, n_actions, alpha=ALPHA, gamma=GAMMA, epsilon=EPS_START, seed=SEED):
        self.q = np.zeros((n_states, n_actions))
        self.alpha, self.gamma, self.epsilon = alpha, gamma, epsilon
        self.n_actions = n_actions
        self.rng = np.random.default_rng(seed)

    def choose_action(self, s):
        """epsilon-greedy: thăm dò với xác suất epsilon, ngược lại khai thác argmax Q(s, a)."""
        if self.rng.random() < self.epsilon:
            return int(self.rng.integers(self.n_actions))
        best = np.flatnonzero(self.q[s] == self.q[s].max())
        return int(self.rng.choice(best))  # phá hòa ngẫu nhiên giữa các hành động bằng nhau

    def update(self, s, a, r, s_next, done):
        """Q(s,a) <- Q(s,a) + alpha * [r + gamma * max_a' Q(s',a') - Q(s,a)]"""
        target = r if done else r + self.gamma * self.q[s_next].max()
        self.q[s, a] += self.alpha * (target - self.q[s, a])

    def decay_epsilon(self):
        self.epsilon = max(EPS_MIN, self.epsilon * EPS_DECAY)


def train(env, agent, episodes=EPISODES, max_steps=MAX_STEPS, log_every=LOG_EVERY):
```

Hình 7.6. Ảnh chụp màn hình ô mã 3 (phần 1/2): siêu tham số và lớp `QLearningAgent`.

Lớp `QLearningAgent` giữ Q-Table dạng mảng NumPy kích thước 144×4, khởi tạo bằng 0. Phương thức `choose_action` hiện thực chính sách ε-greedy;

```
    """Huấn luyện qua nhiều episode; trả về lịch sử phần thưởng, số bước, epsilon, cờ về đích."""
    hist = {"reward": [], "steps": [], "epsilon": [], "success": []}
    t0 = time.perf_counter()
    for ep in range(1, episodes + 1):
        state, total, done = env.reset(), 0.0, False
        s = env.to_index(state)
        for step in range(1, max_steps + 1):
            a = agent.choose_action(s)
            state, r, done = env.step(a)
            s_next = env.to_index(state)
            agent.update(s, a, r, s_next, done)
            s, total = s_next, total + r
            if done:
                break
        hist["reward"].append(total)
        hist["steps"].append(step)
        hist["epsilon"].append(agent.epsilon)
        hist["success"].append(done)
        agent.decay_epsilon()
        if ep % log_every == 0:
            w = slice(ep - log_every, ep)
            print(f"Episode {ep:5d} | epsilon={hist['epsilon'][-1]:.3f} | "
                  f"thưởng TB={np.mean(hist['reward'][w]):8.2f} | bước TB={np.mean(hist['steps'][w]):6.1f} | "
                  f"về đích={100 * np.mean(hist['success'][w]):5.1f}%")
    hist["seconds"] = time.perf_counter() - t0
    return hist


def greedy_path(env, agent, max_steps=MAX_STEPS):
    """Chạy chính sách tham lam pi*(s) = argmax_a Q(s, a) từ S; trả về lộ trình và tổng phần thưởng."""
    state, total, path = env.reset(), 0.0, [env.start]
    for _ in range(max_steps):
        a = int(np.argmax(agent.q[env.to_index(state)]))
        state, r, done = env.step(a)
        total += r
        path.append(state)
        if done:
            break
    return path, total
```

Hình 7.7. Ảnh chụp màn hình ô mã 3 (phần 2/2): hàm `train` và `greedy_path`.

khi nhiều hành động có cùng giá trị lớn nhất (đầu quá trình mọi giá trị đều bằng 0), tác tử chọn ngẫu nhiên giữa chúng để không thiên vị hướng nào. Phương thức `update` là đúng một dòng của phương trình Bellman; ở trạng thái kết thúc, mục tiêu TD chỉ là phần thưởng r.

Đoạn mã 7.3. Lớp `QLearningAgent`: Q-Table, chính sách ε-greedy, cập nhật Bellman và giảm ε.

```
# ag1/qlearning.py
class QLearningAgent:
    """Learning Agent dạng bảng: Q-Table là tri thức, epsilon-greedy là Problem Generator,
    cập nhật Bellman là Learning Element."""

    def __init__(self, n_states, n_actions, alpha=ALPHA, gamma=GAMMA, epsilon=EPS_START, seed=SEED):
        self.q = np.zeros((n_states, n_actions))
        self.alpha, self.gamma, self.epsilon = alpha, gamma, epsilon
        self.n_actions = n_actions
```

```
        self.rng = np.random.default_rng(seed)

    def choose_action(self, s):
        """epsilon-greedy: thăm dò với xác suất epsilon, ngược lại khai thác
        ↪ argmax Q(s, a)."""
        if self.rng.random() < self.epsilon:
            return int(self.rng.integers(self.n_actions))
        best = np.flatnonzero(self.q[s] == self.q[s].max())
        return int(self.rng.choice(best))  # phá hòa ngẫu nhiên giữa các
        ↪ hành động bằng nhau

    def update(self, s, a, r, s_next, done):
        """Q(s,a) <- Q(s,a) + alpha * [r + gamma * max_a' Q(s',a') -
        ↪ Q(s,a)]"""
        target = r if done else r + self.gamma * self.q[s_next].max()
        self.q[s, a] += self.alpha * (target - self.q[s, a])

    def decay_epsilon(self):
        self.epsilon = max(EPS_MIN, self.epsilon * EPS_DECAY)
```

Hàm `train` lặp qua các episode. Mỗi episode bắt đầu tại S; ở mỗi bước tác tử chọn hành động, môi trường trả về trạng thái mới và phần thưởng, tác tử cập nhật Q-Table. Episode dừng khi về đích hoặc khi chạm giới hạn `max_steps`. Sau mỗi episode, ε giảm một bậc; cứ 250 episode hàm in một dòng nhật ký gồm ε, phần thưởng trung bình, số bước trung bình và tỉ lệ về đích.

Đoạn mã 7.4. Hàm `train`: vòng huấn luyện qua các episode và nhật ký tiến trình.

```
# ag1/qlearning.py
def train(env, agent, episodes=EPISODES, max_steps=MAX_STEPS,
↪ log_every=LOG_EVERY):
    """Huấn luyện qua nhiều episode; trả về lịch sử phần thưởng, số bước,
    ↪ epsilon, cờ về đích."""
    hist = {"reward": [], "steps": [], "epsilon": [], "success": []}
    t0 = time.perf_counter()
    for ep in range(1, episodes + 1):
        state, total, done = env.reset(), 0.0, False
        s = env.to_index(state)
        for step in range(1, max_steps + 1):
            a = agent.choose_action(s)
            state, r, done = env.step(a)
            s_next = env.to_index(state)
            agent.update(s, a, r, s_next, done)
            s, total = s_next, total + r
```

```
            if done:
                break
        hist["reward"].append(total)
        hist["steps"].append(step)
        hist["epsilon"].append(agent.epsilon)
        hist["success"].append(done)
        agent.decay_epsilon()
        if ep % log_every == 0:
            w = slice(ep - log_every, ep)
            print(f"Episode {ep:5d} | epsilon={hist['epsilon'][-1]:.3f} | "
                  f"thưởng TB={np.mean(hist['reward'][w]):8.2f} | bước TB={np.mean(hist['steps'][w]):6.1f} | "
                  f"về đích={100 * np.mean(hist['success'][w]):5.1f}%")
    hist["seconds"] = time.perf_counter() - t0
    return hist
```

Sau huấn luyện, `greedy_path` chạy chính sách tham lam $\pi^*(s) = \arg\max_a Q(s,a)$ từ S, không còn thăm dò, để lấy lộ trình tối ưu mà tác tử đã học.

Đoạn mã 7.5. Hàm `greedy_path`: lộ trình theo chính sách tham lam.

```
# ag1/qlearning.py
def greedy_path(env, agent, max_steps=MAX_STEPS):
    """Chạy chính sách tham lam pi*(s) = argmax_a Q(s, a) từ S; trả về lộ trình và tổng phần thưởng."""
    state, total, path = env.reset(), 0.0, [env.start]
    for _ in range(max_steps):
        a = int(np.argmax(agent.q[env.to_index(state)]))
        state, r, done = env.step(a)
        total += r
        path.append(state)
        if done:
            break
    return path, total
```

7.3.4 Khối trực quan hóa và trích Q-Table

Hàm `draw_maze` tô tường lửa, vẽ lộ trình lên lưới; `draw_rewards` vẽ phần thưởng từng episode, đường trung bình trượt 100 episode và lịch ε trên trục phụ. Hàm

[6]:

```python
ACTION_NAMES = ["Trái", "Phải", "Lên", "Xuống"]


def draw_maze(ax, env, path):
    """Vẽ mê cung (tường lửa đỏ cam), lộ trình tối ưu (đường xanh) và hai ô S, G."""
    walls = (env.grid == "#").astype(int)
    ax.imshow(walls, cmap=ListedColormap(["#F8FAFC", "#EA580C"]), vmin=0, vmax=1)
    ys, xs = zip(*path)
    ax.plot(xs, ys, color="#2563EB", lw=4, marker="o", ms=5, label=f"Lộ trình tối ưu ({len(path) - 1} bước)")
    ax.text(*env.start[::-1], "S", ha="center", va="center", fontsize=15, weight="bold", color="#15803D")
    ax.text(*env.goal[::-1], "G", ha="center", va="center", fontsize=15, weight="bold", color="#7C3AED")
    ax.set_xticks(np.arange(-0.5, env.n_cols, 1), minor=True)
    ax.set_yticks(np.arange(-0.5, env.n_rows, 1), minor=True)
    ax.grid(which="minor", color="#94A3B8", lw=0.8)
    ax.grid(which="major", visible=False)
    ax.tick_params(which="minor", length=0)
    ax.set_xticks(range(env.n_cols))
    ax.set_yticks(range(env.n_rows))
    ax.set_title(f"Mê cung {env.n_rows}×{env.n_cols}: lộ trình của tác tử")
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.06), frameon=False)


def draw_rewards(ax, hist, window=100):
    """Phần thưởng mỗi episode, trung bình trượt và epsilon (trục phải)."""
    rewards = np.array(hist["reward"])
    ma = np.convolve(rewards, np.ones(window) / window, mode="valid")
    ep = np.arange(1, len(rewards) + 1)
    ax.plot(ep, rewards, color="#93C5FD", lw=0.8, alpha=0.7, label="Phần thưởng mỗi episode")
    ax.plot(ep[window - 1:], ma, color="#1D4ED8", lw=2.6, label=f"Trung bình trượt {window} episode")
    ax.set_ylim(max(rewards.min(), -400), rewards.max() + 20)
    ax.set_xlabel("Episode")
    ax.set_ylabel("Tổng phần thưởng")
    ax.set_title("Đường cong hội tụ phần thưởng")
    ax2 = ax.twinx()
    ax2.plot(ep, hist["epsilon"], color="#DC2626", lw=2, ls="--", label="epsilon")
    ax2.set_ylabel("epsilon", color="#DC2626")
    ax2.set_ylim(0, 1.05)
    lines = ax.get_lines() + ax2.get_lines()
    ax.legend(lines, [l.get_label() for l in lines], loc="lower right")


def q_table_excerpt(env, agent, path):
    """Trích Q-Table tại các ô then chốt: xuất phát, các điểm rẽ của lộ trình và ô ngay trước đích."""
    keys = [path[0]]
    for prev, cur, nxt in zip(path, path[1:], path[2:]):
        if (cur[0] - prev[0], cur[1] - prev[1]) != (nxt[0] - cur[0], nxt[1] - cur[1]):
            keys.append(cur)
    keys.append(path[-2])
    rows = []
    for s in dict.fromkeys(keys):
        q = agent.q[env.to_index(s)]
        rows.append({"Trạng thái": str(s), **{f"Q({n})": round(float(v), 2) for n, v in zip(ACTION_NAMES, q)},
                     "π*(s)": ACTION_NAMES[int(np.argmax(q))]})
    return pd.DataFrame(rows)
```

Hình 7.8. Ảnh chụp màn hình ô mã 6: các hàm vẽ mê cung, đường cong phần thưởng và trích Q-Table.

`q_table_excerpt` tự chọn các ô then chốt của lộ trình: ô xuất phát, mọi *điểm rẽ* (nơi hướng đi thay đổi) và ô ngay trước đích.

Đoạn mã 7.6. Hàm `q_table_excerpt`: trích Q-Table tại các ô then chốt của lộ trình.

```python
# ag1/viz.py
def q_table_excerpt(env, agent, path):
    """Trích Q-Table tại các ô then chốt: xuất phát, các điểm rẽ của lộ trình và ô ngay trước đích."""
    keys = [path[0]]
    for prev, cur, nxt in zip(path, path[1:], path[2:]):
        if (cur[0] - prev[0], cur[1] - prev[1]) != (nxt[0] - cur[0], nxt[1] - cur[1]):
            keys.append(cur)
    keys.append(path[-2])
    rows = []
    for s in dict.fromkeys(keys):
        q = agent.q[env.to_index(s)]
        rows.append({"Trạng thái": str(s), **{f"Q({n})": round(float(v), 2) for n, v in zip(ACTION_NAMES, q)},
                     "π*(s)": ACTION_NAMES[int(np.argmax(q))]})
    return pd.DataFrame(rows)
```

Đoạn mã 7.7. Hàm `draw_maze`: vẽ mê cung và lộ trình tối ưu.

```python
# ag1/viz.py
def draw_maze(ax, env, path):
    """Vẽ mê cung (tường lửa đỏ cam), lộ trình tối ưu (đường xanh) và hai ô S, G."""
    walls = (env.grid == "#").astype(int)
    ax.imshow(walls, cmap=ListedColormap(["#F8FAFC", "#EA580C"]), vmin=0, vmax=1)
    ys, xs = zip(*path)
    ax.plot(xs, ys, color="#2563EB", lw=4, marker="o", ms=5, label=f"Lộ trình tối ưu ({len(path) - 1} bước)")
    ax.text(*env.start[::-1], "S", ha="center", va="center", fontsize=15, weight="bold", color="#15803D")
    ax.text(*env.goal[::-1], "G", ha="center", va="center", fontsize=15, weight="bold", color="#7C3AED")
    ax.set_xticks(np.arange(-0.5, env.n_cols, 1), minor=True)
    ax.set_yticks(np.arange(-0.5, env.n_rows, 1), minor=True)
    ax.grid(which="minor", color="#94A3B8", lw=0.8)
```

```
    ax.grid(which="major", visible=False)
    ax.tick_params(which="minor", length=0)
    ax.set_xticks(range(env.n_cols))
    ax.set_yticks(range(env.n_rows))
    ax.set_title(f"Mê cung {env.n_rows}×{env.n_cols}: lộ trình của tác tử")
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.06),
     ↪  frameon=False)
```

Đoạn mã 7.8. Hàm `draw_rewards`: đường cong hội tụ phần thưởng và lịch ε.

```
# ag1/viz.py
def draw_rewards(ax, hist, window=100):
    """Phần thưởng mỗi episode, trung bình trượt và epsilon (trục phải)."""
    rewards = np.array(hist["reward"])
    ma = np.convolve(rewards, np.ones(window) / window, mode="valid")
    ep = np.arange(1, len(rewards) + 1)
    ax.plot(ep, rewards, color="#93C5FD", lw=0.8, alpha=0.7, label="Phần
     ↪  thưởng mỗi episode")
    ax.plot(ep[window - 1:], ma, color="#1D4ED8", lw=2.6, label=f"Trung bình
     ↪  trượt {window} episode")
    ax.set_ylim(max(rewards.min(), -400), rewards.max() + 20)
    ax.set_xlabel("Episode")
    ax.set_ylabel("Tổng phần thưởng")
    ax.set_title("Đường cong hội tụ phần thưởng")
    ax2 = ax.twinx()
    ax2.plot(ep, hist["epsilon"], color="#DC2626", lw=2, ls="--",
     ↪  label="epsilon")
    ax2.set_ylabel("epsilon", color="#DC2626")
    ax2.set_ylim(0, 1.05)
    lines = ax.get_lines() + ax2.get_lines()
    ax.legend(lines, [l.get_label() for l in lines], loc="lower right")
```

Chương 8

Kết quả thực nghiệm và phân tích động học học tập

8.1 Ảnh chụp màn hình kết quả thực thi (Copy Screen Output)

Các hình dưới đây là ảnh chụp màn hình thực tế kết quả chạy chương trình trên Jupyter Notebook, đúng yêu cầu của bài tập (copy screen output): nhật ký huấn luyện trên console, lộ trình học được, hai biểu đồ trực quan và Q-Table.

8.1.1 Nhật ký huấn luyện

```
[4]:
env = MazeEnv()
agent = QLearningAgent(env.n_states, env.n_actions)
print(f"Mê cung {env.n_rows}x{env.n_cols}: {env.n_states} trạng thái, {int((env.grid == '#').sum())} ô tường lửa, "
      f"S={env.start}, G={env.goal}")
print(f"alpha={ALPHA}, gamma={GAMMA}, epsilon: {EPS_START} -> {EPS_MIN} (x{EPS_DECAY}/episode), "
      f"{EPISODES} episodes, tối đa {MAX_STEPS} bước/episode")
hist = train(env, agent)
print(f"Huấn luyện xong trong {hist['seconds']:.2f} giây")
```

```
Mê cung 12x12: 144 trạng thái, 55 ô tường lửa, S=(0, 0), G=(11, 11)
alpha=0.1, gamma=0.95, epsilon: 1.0 -> 0.01 (x0.998/episode), 3000 episodes, tối đa 300 bước/episode
Episode   250 | epsilon=0.607 | thưởng TB= -783.52 | bước TB= 185.4 | về đích= 61.6%
Episode   500 | epsilon=0.368 | thưởng TB=  -38.54 | bước TB=  48.7 | về đích=100.0%
Episode   750 | epsilon=0.223 | thưởng TB=   26.85 | bước TB=  34.7 | về đích=100.0%
Episode  1000 | epsilon=0.135 | thưởng TB=   50.51 | bước TB=  29.7 | về đích=100.0%
Episode  1250 | epsilon=0.082 | thưởng TB=   62.76 | bước TB=  26.9 | về đích=100.0%
Episode  1500 | epsilon=0.050 | thưởng TB=   68.76 | bước TB=  25.7 | về đích=100.0%
Episode  1750 | epsilon=0.030 | thưởng TB=   71.50 | bước TB=  25.2 | về đích=100.0%
Episode  2000 | epsilon=0.018 | thưởng TB=   74.09 | bước TB=  24.6 | về đích=100.0%
Episode  2250 | epsilon=0.011 | thưởng TB=   74.95 | bước TB=  24.5 | về đích=100.0%
Episode  2500 | epsilon=0.010 | thưởng TB=   75.36 | bước TB=  24.3 | về đích=100.0%
Episode  2750 | epsilon=0.010 | thưởng TB=   76.02 | bước TB=  24.3 | về đích=100.0%
Episode  3000 | epsilon=0.010 | thưởng TB=   76.24 | bước TB=  24.3 | về đích=100.0%
Huấn luyện xong trong 2.39 giây
```

Hình 8.1. Ảnh chụp màn hình kết quả ô 4: cấu hình bài toán và nhật ký huấn luyện sau mỗi 250 episode.

8.1.2 Lộ trình tối ưu

Tác tử tìm được lộ trình 24 bước với tổng phần thưởng 77, đúng bằng độ dài đường ngắn nhất 24 bước do BFS tìm trên bản đồ. Tổng phần thưởng khớp với

```
[5]: path, total = greedy_path(env, agent)
     print(f"Lộ trình học được: {len(path) - 1} bước, tổng phần thưởng {total:+.1f}")
     print(f"Đường ngắn nhất (BFS kiểm chứng): {env.shortest_path_length()} bước")
     print(" -> ".join(str(p) for p in path))

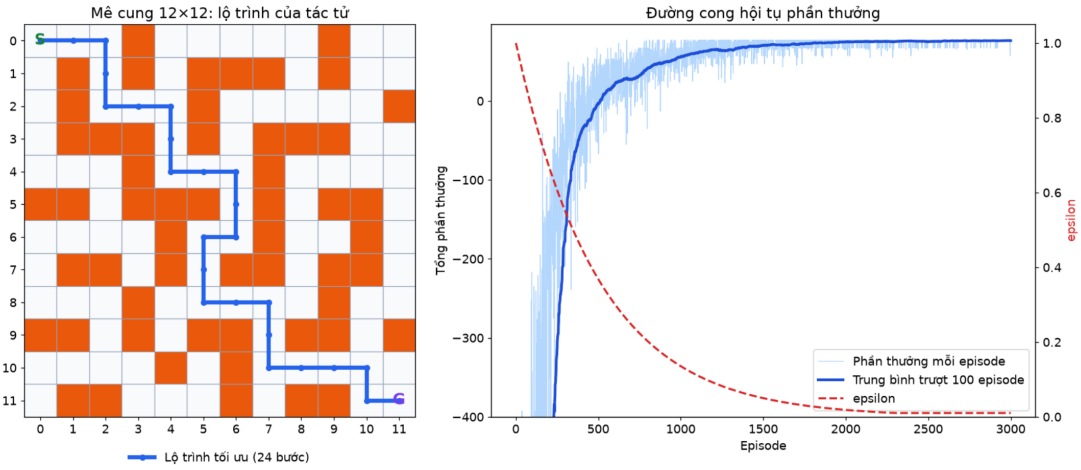
Lộ trình học được: 24 bước, tổng phần thưởng +77.0
Đường ngắn nhất (BFS kiểm chứng): 24 bước
(0, 0) -> (0, 1) -> (0, 2) -> (1, 2) -> (2, 2) -> (2, 3) -> (2, 4) -> (3, 4) -> (4, 4) -> (4, 5) -> (4, 6) -> (5, 6) -> (6, 6) -
> (6, 5) -> (7, 5) -> (8, 5) -> (8, 6) -> (8, 7) -> (9, 7) -> (10, 7) -> (10, 8) -> (10, 9) -> (10, 10) -> (11, 10) -> (11, 11)
```

Hình 8.2. Ảnh chụp màn hình kết quả ô 5: lộ trình học được, đối chiếu với đường ngắn nhất tìm bằng BFS.

tính toán: 24 bước gồm 23 bước thường (-1 mỗi bước) và bước cuối vào đích ($+100$), tức $-23 + 100 = 77$.

8.1.3 Biểu đồ trực quan

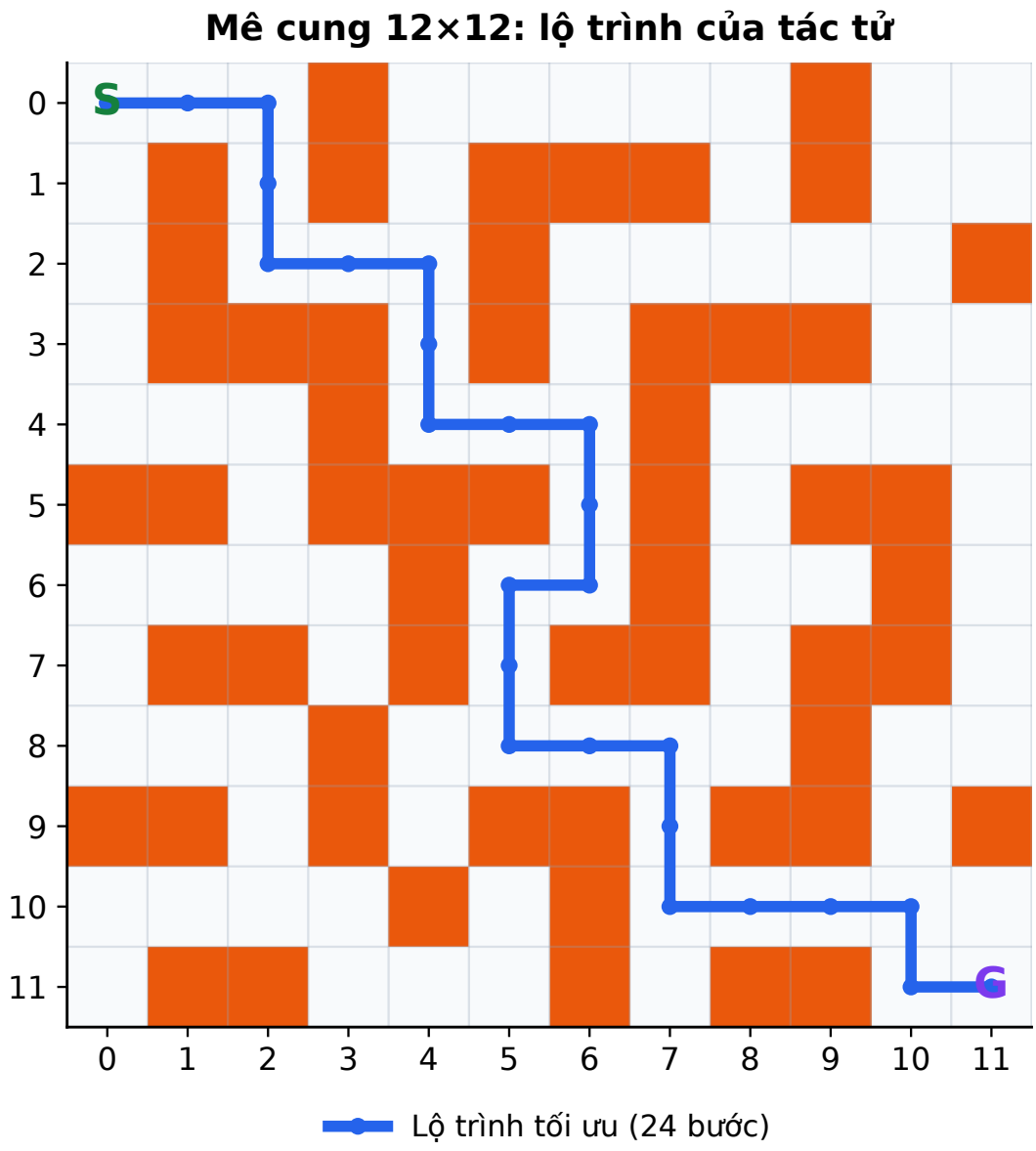
```
[7]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6.4), gridspec_kw={"width_ratios": [1, 1.35]})
     draw_maze(ax1, env, path)
     draw_rewards(ax2, hist)
     fig.tight_layout()
     plt.show()
```



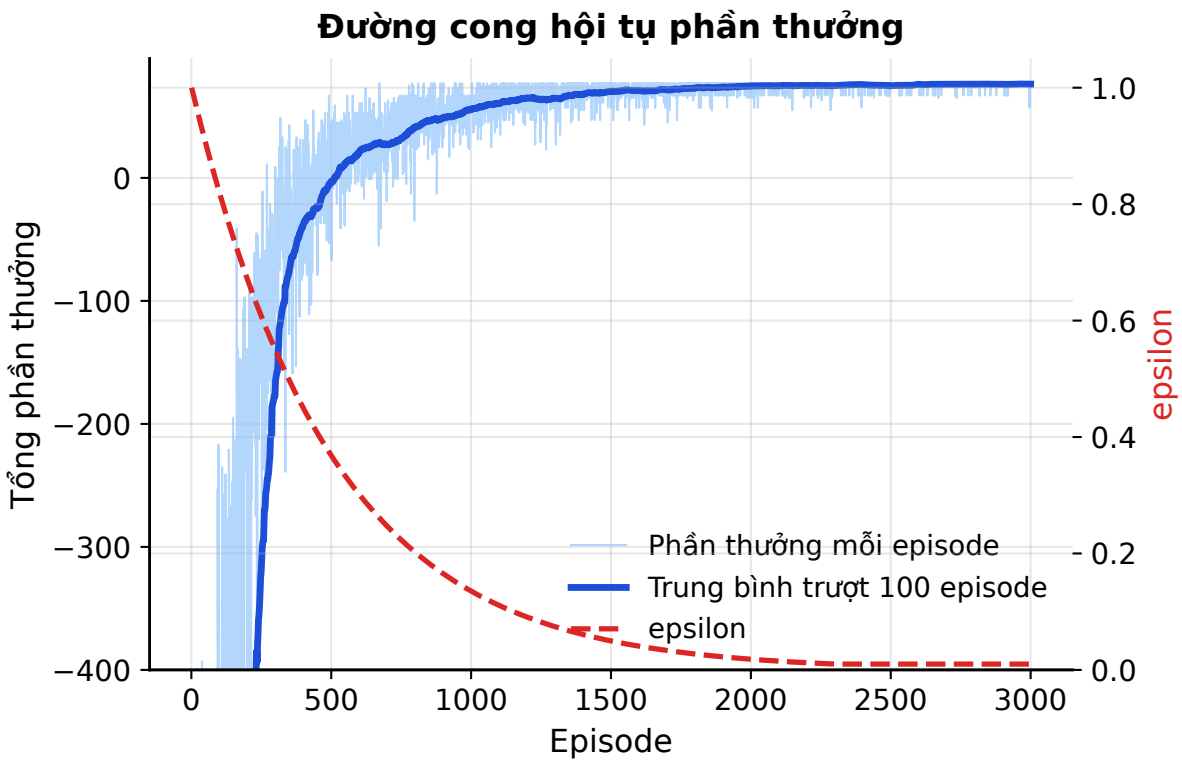
Hình 8.3. Ảnh chụp màn hình kết quả ô 7: lộ trình tối ưu trong mê cung 12×12 và đường cong hội tụ phần thưởng sau 3.000 episode.

Để đọc rõ hơn, hai biểu đồ trong Hình 8.3 được vẽ lại dạng vector ở cỡ toàn trang từ chính số liệu notebook đã lưu (Hình 8.4 và Hình 8.5).

8.1.4 Q-Table và lưu số liệu



Hình 8.4. Lộ trình tối ưu 24 bước của tác tử trong mê cung (vẽ lại dạng vector).



Hình 8.5. Phần thưởng mỗi episode, trung bình trượt 100 episode và lịch giảm ε (vẽ lại dạng vector).

```
[8]: 1 qtab = q_table_excerpt(env, agent, path)
     2 qtab
```

[8]:

	Trạng thái	Q(Trái)	Q(Phải)	Q(Lên)	Q(Xuống)	π*(s)
0	(0, 0)	6.04	16.88	6.04	13.28	Phải
1	(0, 2)	16.88	9.82	9.82	20.87	Xuống
2	(2, 2)	14.01	25.28	20.87	14.02	Phải
3	(2, 4)	25.28	18.67	25.28	30.17	Xuống
4	(4, 4)	23.81	35.59	30.17	23.81	Phải
5	(4, 6)	35.59	29.52	35.59	41.60	Xuống
6	(6, 6)	48.26	35.84	41.60	35.84	Trái
7	(6, 5)	39.26	44.84	39.26	51.85	Xuống
8	(8, 5)	51.85	59.61	51.85	46.63	Phải
9	(8, 7)	59.61	59.61	54.80	68.21	Xuống
10	(10, 7)	63.85	77.74	68.21	68.21	Phải
11	(10, 10)	82.88	94.00	82.88	94.00	Xuống
12	(11, 10)	85.00	100.00	88.30	85.00	Phải

Hình 8.6. Ảnh chụp màn hình kết quả ô 8: trích đoạn Q-Table tại các ô then chốt của lộ trình.

[9]:
```python
def first_episode(cond):
    return next((i + 1 for i, ok in enumerate(cond) if ok), None)

rewards = np.array(hist["reward"])
ma = np.convolve(rewards, np.ones(100) / 100, mode="valid")
metrics = {
    "params": {"alpha": ALPHA, "gamma": GAMMA, "eps_start": EPS_START, "eps_min": EPS_MIN, "eps_decay": EPS_DECAY,
               "episodes": EPISODES, "max_steps": MAX_STEPS, "r_fire": R_FIRE, "r_goal": R_GOAL, "r_step": R_STEP,
               "n_rows": env.n_rows, "n_cols": env.n_cols, "n_states": env.n_states,
               "n_fire": int((env.grid == "#").sum()), "start": str(env.start), "goal": str(env.goal)},
    "result": {"path_len": len(path) - 1, "bfs_len": env.shortest_path_length(), "greedy_reward": total,
               "seconds": hist["seconds"], "first_success": first_episode(hist["success"]),
               "ma_first_positive": first_episode(ma > 0) + 99,
               "eps_below_005": first_episode(np.array(hist["epsilon"]) < 0.05),
               "reward_first100": rewards[:100].mean(), "reward_min": rewards.min(),
               "reward_last100": rewards[-100:].mean(), "steps_last100": np.mean(hist["steps"][-100:]),
               "success_first250": np.mean(hist["success"][:250]), "success_last100": np.mean(hist["success"][-100:])},
    "qtable": qtab.to_dict(orient="records"),
    "history": {k: hist[k] for k in ("reward", "steps", "epsilon", "success")},
    "path": [list(p) for p in path],
}
out = Path("../outputs/metrics/ch8_qlearning.json")
out.parent.mkdir(parents=True, exist_ok=True)
out.write_text(json.dumps(metrics, ensure_ascii=False, indent=1, default=lambda o: o.item()), encoding="utf-8")
print("Đã ghi", out, "-", {k: metrics["result"][k] for k in ("path_len", "bfs_len", "greedy_reward")})
```

```
Đã ghi ..\outputs\metrics\ch8_qlearning.json - {'path_len': 24, 'bfs_len': 24, 'greedy_reward': 77.0}
```

Hình 8.7. Ảnh chụp màn hình ô 9: lưu số liệu thực nghiệm ra tệp JSON cho báo cáo.

8.2 Phân tích động học học tập và hội tụ chính sách

Bảng 8.1 tổng hợp nhật ký huấn luyện theo từng khối 250 episode. Kết hợp với Hình 8.5, quá trình tự học của tác tử chia thành hai pha rõ rệt.

Bảng 8.1. Diễn biến huấn luyện theo từng khối 250 episode.

Episode	ε cuối khối	Thưởng TB 250 ep.	Số bước TB	Về đích (%)
250	0,607	−783,52	185,4	61,6
500	0,368	−38,54	48,7	100,0
750	0,223	26,85	34,7	100,0
1.000	0,135	50,51	29,7	100,0
1.250	0,082	62,76	26,9	100,0
1.500	0,050	68,76	25,7	100,0
1.750	0,030	71,50	25,2	100,0
2.000	0,018	74,09	24,6	100,0
2.250	0,011	74,95	24,5	100,0
2.500	0,010	75,36	24,3	100,0
2.750	0,010	76,02	24,3	100,0
3.000	0,010	76,24	24,3	100,0

8.2.1 Pha thăm dò (khoảng 500 episode đầu)

Với ε gần 1, tác tử gần như đi ngẫu nhiên. Nó liên tục đâm vào tường lửa, mỗi lần mất 10 điểm và đứng yên, nên phần thưởng rất thấp: trung bình 100 episode đầu chỉ -1.302, episode tệ nhất -1.704. Nhiều episode chạm giới hạn 300 bước mà không về đích; tỉ lệ về đích của 250 episode đầu là 61,60%. Lần đầu tiên tác tử tình cờ về được đích là ở episode 18. Từ đó phần thưởng $+100$ bắt đầu được lan ngược dần từ ô đích về các ô xa hơn qua từng lần cập nhật Bellman. Đây là giai đoạn *Problem Generator* hoạt động mạnh nhất: tác tử tích lũy kinh nghiệm chứ chưa khai thác.

8.2.2 Pha khai thác và hội tụ

Khi giá trị lan đủ xa, các hành động tốt nổi lên trong Q-Table và ε giảm làm tác tử ngày càng đi theo chúng. Đường trung bình trượt 100 episode chuyển sang

dương ở khoảng episode 513; từ khối 500 trở đi tác tử về đích 100% số episode. ε xuống dưới 0,05 ở episode 1.498; sau đó phần thưởng ổn định gần trần 77. Trung bình 100 episode cuối đạt 76,22 với 24,24 bước. Con số này thấp hơn 77 một chút vì ε vẫn giữ ở mức tối thiểu 0,01, thỉnh thoảng tác tử còn đi một bước ngẫu nhiên. Khi tắt thăm dò (chính sách tham lam), tác tử đi đúng lộ trình tối ưu 24 bước.

8.3 Phân tích Q-Table và chính sách hành động tối ưu

Bảng 8.2 trích Q-Table tại ô xuất phát, các điểm rẽ của lộ trình và ô ngay trước đích; giá trị lớn nhất mỗi hàng được in đậm và quyết định hành động $\pi^*(s)$.

Bảng 8.2. Trích đoạn Q-Table và chính sách hành động tối ưu $\pi^*(s)$ tại các ô then chốt.

Trạng thái s	Q(Trái)	Q(Phải)	Q(Lên)	Q(Xuống)	$\pi^*(s)$	Ý nghĩa điều hướng
(0, 0)	6,04	**16,88**	6,04	13,28	Phải	Xuất phát: chọn Phải; Trái, Lên là tường/biên
(0, 2)	16,88	9,82	9,82	**20,87**	Xuống	Điểm rẽ: Xuống sang (1, 2); Phải, Lên là tường/biên
(2, 2)	14,01	**25,28**	20,87	14,02	Phải	Điểm rẽ: Phải sang (2, 3); Trái, Xuống là tường/biên
(2, 4)	25,28	18,67	25,28	**30,17**	Xuống	Điểm rẽ: Xuống sang (3, 4); Phải là tường/biên
(4, 4)	23,81	**35,59**	30,17	23,81	Phải	Điểm rẽ: Phải sang (4, 5); Trái, Xuống là tường/biên
(4, 6)	35,59	29,52	35,59	**41,60**	Xuống	Điểm rẽ: Xuống sang (5, 6); Phải là tường/biên
(6, 6)	**48,26**	35,84	41,60	35,84	Trái	Điểm rẽ: Trái sang (6, 5); Phải, Xuống là tường/biên
(6, 5)	39,26	44,84	39,26	**51,85**	Xuống	Điểm rẽ: Xuống sang (7, 5); Trái, Lên là tường/biên
(8, 5)	51,85	**59,61**	51,85	46,63	Phải	Điểm rẽ: Phải sang (8, 6); Xuống là tường/biên
(8, 7)	59,61	59,61	54,80	**68,21**	Xuống	Điểm rẽ: Xuống sang (9, 7); Lên là tường/biên
(10, 7)	63,85	**77,74**	68,21	68,21	Phải	Điểm rẽ: Phải sang (10, 8); Trái là tường/biên
(10, 10)	82,88	**94,00**	82,88	94,00	Xuống	Điểm rẽ: Xuống sang (11, 10)
(11, 10)	85,00	**100,00**	88,30	85,00	Phải	Bước quyết định: Phải vào đích G; Trái, Xuống là tường/biên

Các giá trị trong bảng không phải con số tùy ý. Chúng hội tụ về đúng tổng phần thưởng chiết khấu mà phương trình Bellman dự đoán, và có thể kiểm tra bằng tay:

- **Ô ngay trước đích** $(11, 10)$**:** $Q(\text{Phải}) = 100$, đúng bằng R_{goal} vì bước này kết thúc episode, không còn phần thưởng tương lai.
- **Ô** $(10, 10)$**:** cả Phải và Xuống đều bằng $94 = -1 + 0{,}95 \times 100$. Hai hướng cùng dẫn tới một ô kề đích nên ngang giá trị; cả hai đều tối ưu.
- **Ô xuất phát** $(0, 0)$**:** $Q(\text{Phải}) = 16{,}88$ là giá trị của lộ trình 24 bước: $-\sum_{k=0}^{22} 0{,}95^k + 0{,}95^{23} \times 100 \approx -13{,}85 + 30{,}73 = 16{,}88$.
- **Hành động đâm tường:** tại $(0, 0)$, Trái và Lên đưa robot ra khỏi mê cung nên robot đứng yên: $Q = -10 + 0{,}95 \times 16{,}88 = 6{,}04$. Giá trị này thấp hơn hẳn hướng tốt, nên tác tử đã học được cách tránh tường lửa.

Giá trị Q dọc lộ trình tăng dần từ 16,88 ở S lên 100 ở sát đích, mỗi bước nhân thêm γ và cộng phần thưởng bước. Q-Table vì thế đóng vai một "bản đồ độ dốc": từ bất kỳ ô nào trên lộ trình, đi theo hướng có Q lớn nhất là đi về đích nhanh nhất.

Chương 9

Sự giao thoa giữa LLM Agent và học tăng cường

9.1 Học tăng cường từ phản hồi con người (RLHF)

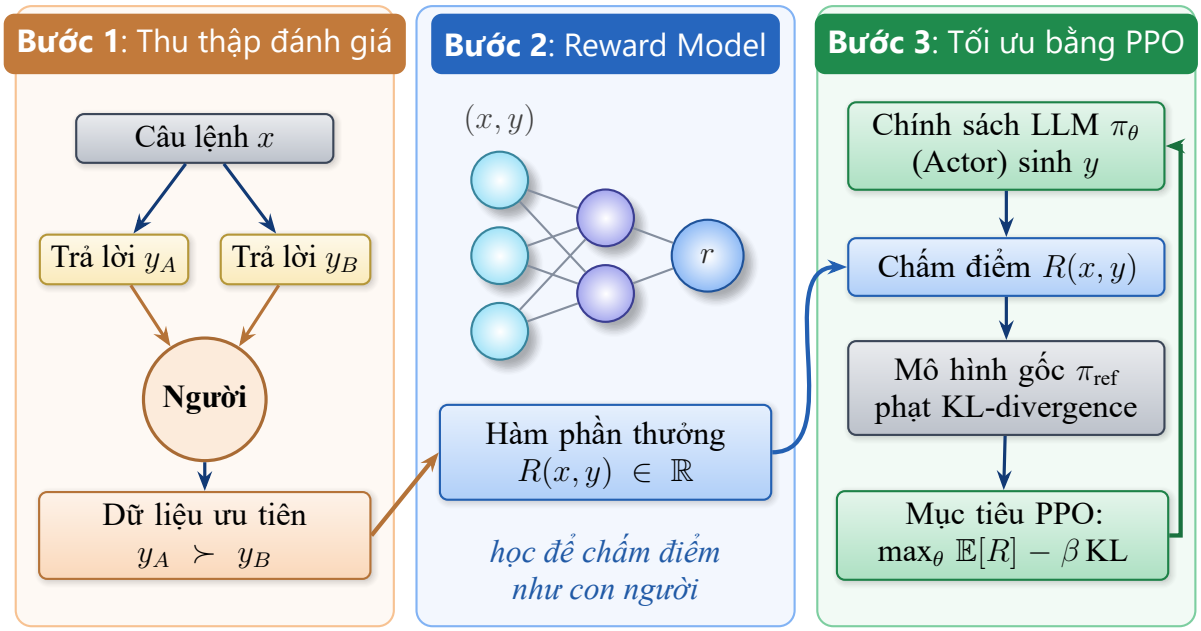
Các tác tử AI hiện đại dựa trên LLM liên hệ chặt chẽ với học tăng cường qua kỹ thuật **RLHF** (Reinforcement Learning from Human Feedback) [9]. Quy trình gồm ba bước (Hình 9.1):

1. **Huấn luyện mô hình phần thưởng (Reward Model).** Thu thập đánh giá ưu tiên của con người giữa các câu trả lời của mô hình (câu nào tốt hơn), từ đó huấn luyện một hàm phần thưởng $R_\phi(x, y)$ chấm điểm câu trả lời y cho câu hỏi x.
2. **Tối ưu chính sách bằng PPO.** Dùng thuật toán Actor-Critic PPO (Proximal Policy Optimization) cập nhật trọng số LLM để tối đa hóa điểm của Reward Model, đồng thời phạt độ lệch KL so với mô hình gốc để mô hình không "lách luật" phần thưởng:

$$\max_{\theta} \; \mathbb{E}_{x,\, y\sim\pi_\theta}[R_\phi(x, y)] - \beta\, \mathrm{KL}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)) \,.$$

3. **Định hình hành vi an toàn.** RLHF biến một mô hình dự đoán từ kế tiếp thô thành một tác tử hội thoại biết từ chối yêu cầu độc hại, tuân thủ chỉ dẫn và giải trình mạch lạc.

Có thể đặt RLHF vào khung MDP của Chương 4: trạng thái là câu hỏi cùng phần câu trả lời đã sinh, hành động là từ kế tiếp, phần thưởng do Reward Model chấm khi câu trả lời hoàn tất. Đây là cùng một vòng lặp tác tử – môi trường mà tác tử Q-Learning của bài thực hành sử dụng, chỉ khác ở quy mô.

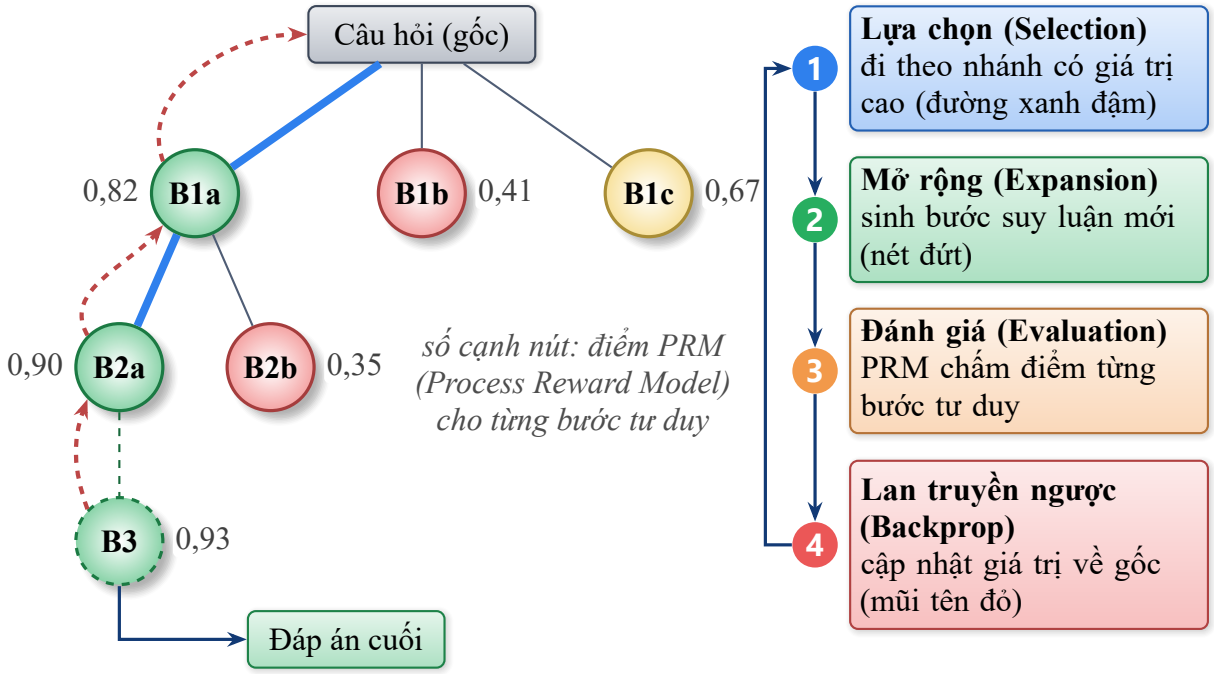


Hình 9.1. Quy trình ba bước của RLHF: dữ liệu ưu tiên của con người, mô hình phần thưởng và tối ưu chính sách bằng PPO có ràng buộc KL.

9.2 Tác tử tự hoàn thiện và suy luận theo cây MCTS

Các hệ tác tử tiên tiến đưa thuật toán học tăng cường kinh điển vào chính quá trình suy luận (inference-time compute):

- **Monte Carlo Tree Search (MCTS) và Q* Search.** Thay vì sinh một luồng suy luận duy nhất, tác tử khám phá một cây suy luận nhiều nhánh qua bốn bước chọn – mở rộng – đánh giá – lan truyền ngược. Một mô hình đánh giá từng bước (Process Reward Model – PRM) chấm điểm mỗi bước tư duy, đóng vai hàm giá trị giống như $Q(s, a)$ trong Q-Learning (Hình 9.2).
- **Tự sửa sai (Self-Correction).** Tác tử tự tạo môi trường phản hồi (self-play): viết mã, chạy kiểm thử đơn vị, đọc lỗi và sửa, lặp lại cho tới khi đạt mục tiêu. Kết quả kiểm thử chính là tín hiệu phần thưởng.

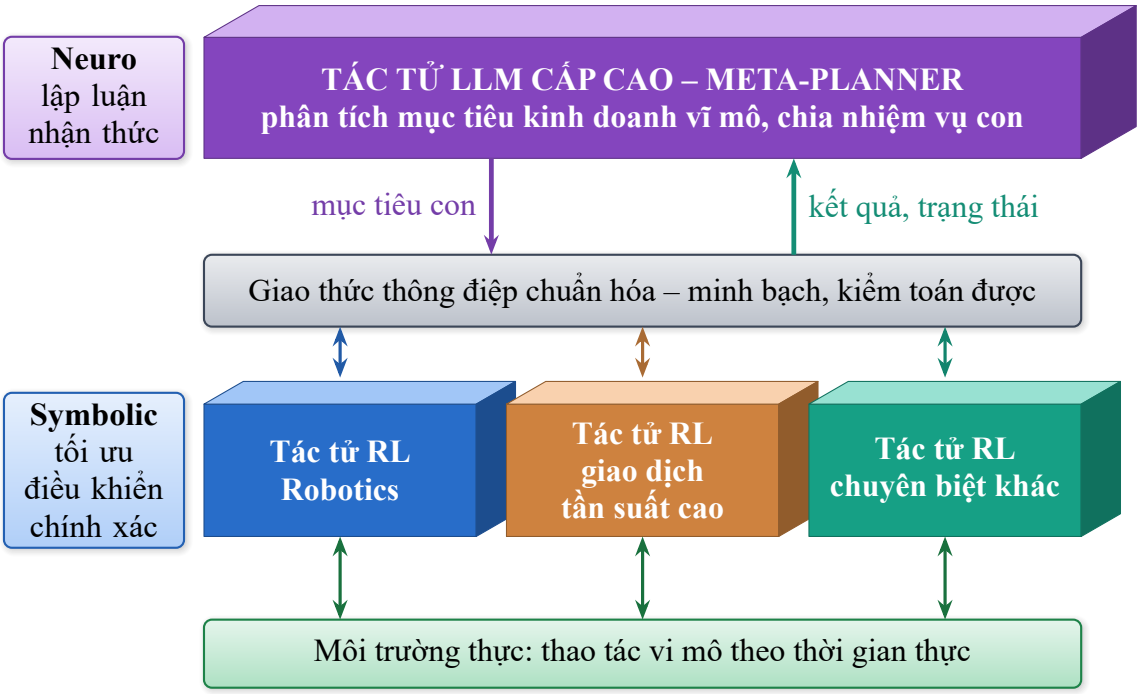


Hình 9.2. Suy luận dạng cây MCTS với mô hình đánh giá từng bước (PRM).

9.3 Xu thế hội tụ Neuro-Symbolic trong hệ đa tác tử

Kết hợp năng lực lập luận ngôn ngữ của LLM (Neuro) với các giải thuật tối ưu hóa điều khiển chính xác của RL và quy hoạch động (Symbolic/Dynamic Programming) mở ra các hệ thống tự hành mới:

- **Phân tầng điều khiển (Hierarchical Agent Control).** Tác tử LLM cấp cao làm Meta-Planner: phân tích mục tiêu nghiệp vụ, chia thành mục tiêu con và giao cho các tác tử RL chuyên biệt thực thi thao tác vi mô thời gian thực (robot, giao dịch tần suất cao) (Hình 9.3).
- **Trao đổi thông điệp chuẩn hóa.** Các tác tử trong Agentic Mesh tương tác qua giao thức mở, bảo đảm tính minh bạch và khả năng kiểm toán toàn diện như yêu cầu quản trị ở Chương 6.



Hình 9.3. Phân tầng điều khiển: tác tử LLM Meta-Planner giao mục tiêu con cho các tác tử RL chuyên biệt.

Chương 10

Tổng kết và đánh giá học thuật

10.1 Đánh giá tổng quan về năng lực và giới hạn hiện tại của AI Agent

Tổng hợp từ Google Cloud [1], IBM Think [2], McKinsey [3] và GeeksforGeeks [4], có thể khẳng định AI Agent là bước chuyển quan trọng của trí tuệ nhân tạo: từ vai trò trợ lý thụ động trả lời câu hỏi sang thực thể tự hành giải quyết công việc thực tế. Ba trụ cột suy luận, bộ nhớ và công cụ cho phép tác tử lập kế hoạch, ghi nhớ và tác động vào hệ thống; các mô hình suy luận như ReAct, ReWOO, Reflection quyết định sự cân bằng giữa độ linh hoạt và chi phí; mạng lưới tác tử (Agentic Mesh) mở rộng mô hình này ra toàn doanh nghiệp.

Tuy vậy, các thách thức về độ tin cậy, chi phí token, an toàn quyền hạn (privilege escalation) và nguy cơ vòng lặp vô hạn đòi hỏi các rào chắn nghiêm ngặt: nhật ký kiểm toán, định danh tác tử, ngưỡng giới hạn (guardrails) và cơ chế con người can thiệp (human-in-the-loop). Mức tự trị của tác tử càng cao thì các cơ chế này càng cần thiết.

10.2 Bài học kinh nghiệm từ bài thực hành lập trình Q-Learning

Qua quá trình lập trình và chạy tác tử Q-Learning trong mê cung 12×12:

- **Sức mạnh của cập nhật Bellman.** Tác tử không biết bản đồ ban đầu, nhưng sau 3.000 episode tương tác (2,393 giây trên CPU) đã tự tìm ra lộ trình 24 bước, đúng bằng đường ngắn nhất do BFS kiểm chứng. Các giá trị Q-Table hội tụ về đúng tổng phần thưởng chiết khấu mà phương trình Bellman dự đoán.
- **Tầm quan trọng của cân bằng thăm dò – khai thác.** Giai đoạn đầu với ε lớn có phần thưởng rất thấp nhưng cần thiết, vì nhờ đó tác tử tình cờ tìm thấy

đích và lan giá trị ngược về. Nếu ε giảm quá nhanh, tác tử dễ mắc kẹt ở một lộ trình chưa tối ưu; nếu ε luôn lớn, chính sách không thể ổn định.

- **Thiết kế phần thưởng định hình hành vi.** Hình phạt -1 mỗi bước khiến tác tử ưu tiên đường ngắn; hình phạt -10 khi đâm tường dạy nó tránh tường lửa. Đây chính là vai trò của "tiêu chuẩn hiệu năng" trong kiến trúc Learning Agent.
- **Guardrail cũng cần cho tác tử nhỏ.** Giới hạn 300 bước mỗi episode ngăn tác tử lặp vô hạn ở giai đoạn đi ngẫu nhiên, đúng như nguyên tắc `max_iterations` mà IBM khuyến nghị.

10.3 Hướng phát triển trong các bài tập kế tiếp

Mở rộng kiến trúc tác tử kết hợp đồ thị tri thức (Knowledge Graph RAG) và tích hợp công cụ Function Calling cho các bài toán tài chính và y tế phức hợp trong các bài thực hành tiếp theo của học phần.

Tài liệu tham khảo

[1] Google Cloud. *What are AI agents? Definition, examples, and types*. 2024. URL: https://cloud.google.com/discover/what-are-ai-agents (visited on 10/23/2026).

[2] IBM Think. *What are AI agents?* 2024. URL: https://www.ibm.com/think/topics/ai-agents (visited on 10/23/2026).

[3] McKinsey & Company. *What is an AI agent? McKinsey Explainers*. 2024. URL: https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-an-ai-agent (visited on 10/23/2026).

[4] GeeksforGeeks. *What is Reinforcement Learning?* 2024. URL: https://www.geeksforgeeks.org/machine-learning/what-is-reinforcement-learning/ (visited on 10/23/2026).

[5] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020.

[6] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations (ICLR)*. 2023. URL: https://arxiv.org/abs/2210.03629.

[7] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*. 2023. arXiv: 2305.18323. URL: https://arxiv.org/abs/2305.18323.

[8] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press, 2018.

[9] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. "Deep Reinforcement Learning from Human Preferences". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017. URL: https://arxiv.org/abs/1706.03741.